A Project Report on

A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology
in
Artificial Intelligence and Data Science

Submitted By

N. VENKATA SUBBAIAH (22705A3010)
G. NAGA HARSHITHA (22701A3055)
G. VEDAVATHI (22701A3094)
B. NAGA THARUN KUMAR (23701A3057)

Under the esteemed guidance of

Miss. D. SAI SHIREESHA MCA
Assistant Professor
Department of Artificial Intelligence and Data Science



Department of Artificial Intelligence and Data Science
Annamacharya Institute of Technology and Sciences
(Autonomous)
New Boyanapalli, Rajampet – 516126, Annamayya (Dist.), A.P.
(Affiliated To J.N.T.U, Anantapur)

2025-2026

Department of Artificial Intelligence and Data Science

Annamacharya Institute of Technology and Sciences

(Autonomous)

(Affiliated to J.N.T.U, Anantapur)

New Boyanapalli, Rajampet – 516126, Annamayya (Dist.), A.P.



CERTIFICATE

This is to certify that the project entitled **"A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques"** is submitted by

N. VENKATA SUBBAIAH	**(22705A3010)**
G. NAGA HARSHITHA	**(22701A3055)**
G. VEDAVATHI	**(22701A3094)**
B. NAGA THARUN KUMAR	**(22701A3057)**

in partial fulfillment of requirements for the award of Degree of **Bachelor of Technology** in "**Artificial Intelligence and Data Science"**, is a record of bonafide work carried out by them during the academic year 2025-2026.

Project Guide

Miss. D. Sai Shireesha MCA

Assisstant Professor

Department of AI&DS

AITS, Rajampet

Head of the Department

Dr. P. Phanindra Kumar Reddy M.Tech,Ph.D.

Associate Professor & Head

Department of AI&DS

AITS, Rajampet

Department of Artificial Intelligence and Data Science

Annamacharya Institute of Technology and Sciences (Autonomous)

(Affiliated to J.N.T.U, Anantapur)

New Boyanapalli, Rajampet – 516126, Annamayya (Dist.), A.P.

CERTIFICATE

This is to certify that the project entitled "**A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques**" is submitted by

N. VENKATA SUBBAIAH	**(23705A3010)**
G. NAGA HARSHITHA	**(22701A3055)**
G. VEDAVATHI	**(22701A3094)**
B. NAGA THARUN KUMAR	**(22701A3057)**

in partial fulfillment of requirements for the award of Degree of **Bachelor of Technology** in **"Artificial Intelligence and Data Science"**, is a record of bonafide work carried out by them during the academic year 2025-2026.

Project viva-voce held on: _____

Internal Examiner External Examiner

Annamacharya Institute of Technology and Sciences::Rajampet
(Autonomous)

Research and Development Cell

Department of Artificial Intelligence and Data Science

ANTI-PLAGIARISM CERTIFICATE

This is to certify that the project/dissertation report titled **"A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques"** is submitted by

Mr. N. VENKATA SUBBAIAH	**With H.T. No. (23705A3010)**
Ms. G. NAGA HARSHITHA	**With H.T. No. (22701A3055)**
Ms. G. VEDAVATHI	**With H.T. No. (22701A3094)**
Mr. B. NAGA THARUN KUMAR	**With H.T. No. (22715A3057)**

in partial fulfillment of **Bachelor of Technology** in the Department of **Artificial Intelligence and Data Science** contains a Plagiarism of _____ % which is within the acceptable limits.

Date:

Dean/Coordinator

Research & Development Cell

DECLARATION

We hereby declare that the project report entitled "**A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques**" under the guidance of **Miss. D. Sai Shireesha** MCA. Assistant Professor, Department of Artificial Intelligence & Data Science is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science.

This is a record of bonafide work carried out by us and the results embodied in project report have not been reproduced or copied from any source. The results embodied in this project report have been submitted to any other university or institute for the Award of any other Degree or Diploma.

Project Associates:

N. VENKATA SUBBAIAH

G. NAGA HARSHITHA

G. VEDAVATHI

B. NAGA THARUN KUMAR

ACKNOWLEDGEMENT

An endeavour of a long period can be successful only with the advice of many well-wishers. We take this opportunity to express our deep gratitude and appreciation to all those who encouraged us for successfully completion of the project work.

Our heartfelt thanks to our Guide, **Miss. D. Sai Shireesha,** Department of Artificial Intelligence and Data Science, Annamacharya Institute of Technology and Sciences, Rajampet, for his valuable guidance and suggestions in analyzing and testing throughout the period, till the end of the project work completion.

We wish to express sincere thanks and gratitude to **Dr. P. Phanindra Kumar Reddy,** Head of the Department of Artificial Intelligence and Data Science, for his encouragement and facilities that were offered to us for carrying out this project.

We take this opportunity to offer gratefulness to our Principal **Dr. S. M. V. Narayana,** for providing all sorts of help during the project work.

We would like to place on record, our grateful thanks to **Dr. E. Saibaba Reddy,** Vice Chancellor, Annamacharya University for providing all the facilities and help in carrying out the entire project.

We are very much thankful to **Dr. C. Gangi Reddy**, Honorary Secretary of the Annamacharya Educational Trust, for his help in providing good facilities in our college.

We would express our sincere thanks to all faculty members of **Artificial Intelligence and Data Science Department, batch-mates, friends and lab-technicians**, who have helped us to complete the project work successfully.

Finally, we express our sincere thanks to **our parents** who has provided their heartfelt support and encouragement in the accomplishment to complete this project successfully.

Project Associates:
N. VENKATA SUBBAIAH
G. NAGA HARSHITHA
G. VEDAVATHI
B. NAGA THARUN KUMAR

ABSTRACT

Paraphrase identification is a fundamental task in Natural Language Processing (NLP) that aims to determine whether two sentences express the same meaning using different wording. This classification-based project explores various deep learning architectures to classify sentence pairs into two categories: paraphrase or non-paraphrase. Initially, existing models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT are employed to benchmark performance. Subsequently, more advanced transformer-based models like and RoBERTa are proposed to enhance accuracy and semantic understanding. These models leverage pre-trained contextual embeddings and contrastive learning for improved textual similarity detection. The project uses benchmark datasets suitable for sentence-pair classification to train and evaluate the models. While this work is academic in nature, it simulates accurate challenges such as ambiguity, syntax variation, and context preservation. The goal is to understand the strengths and limitations of various models in paraphrase detection and to compare their performance comprehensively. This exploration can contribute to applications such as question-answer matching, duplicate detection, and content summarization systems.

Keywords: Paraphrase Identification, NLP, Sentence Similarity, Classification, Deep Learning, , RoBERTa, BERT, CNN, RNN

TABLE OF CONTENTS

TITLE	PAGE No
Chapter 1 INTRODUCTION	**1-3**
1. Motivation	1-2
2. Definition	2-3
Chapter 2 LITERATURE SURVEY	**4-6**
Chapter 3 SYSTEM ANALYSIS	**7-11**
3.1 Existing System	8
3.1.1 Disadvantages of existing system	8-9
3.2 Proposed System	9
3.2.1 Advantages of Proposed System	9-10
3.3 Models	10-11
3.4 Modules Used in Proposed System	11
3.4.1 User	11
3.4.2 System	11
3.5 Algorithms Used	12
Chapter 4 SYSTEM REQUIREMENTS SPECIFICATIONS	**13-16**
4.1 Software Requirements	13
4.2 Hardware Requirements	13
4.3 Feasibility Study	13
4.3.1 Economical Feasibility	14
4.3.2 Technical Feasibility	14
4.3.3 Behavioural Feasibility	15
4.4 Function and Non-Functional Requirements	15-16
Chapter 5 SYSTEM DESIGN	**17-29**
5.1 Architecture Design	17-19
5.2 Introduction to UML Diagrams	19-20
5.2.1 Goals	20

5.3 UML Notations 21-22
5.4 UML Diagrams 23
5.4.1 Use Case Diagram 23
5.4.2 Class Diagram 24
5.4.3 Sequence Diagram 25
5.4.4 Deployment Diagram 26
5.4.5 Activity Diagram 27
5.4.6 Component Diagram 28
5.4.7 ER Diagram 29
Chapter 6 SYSTEM CODING AND IMPLEMENTATION 30-37
6.1 Introduction to Python Programming Language 30
6.2 Benefits of Python 31
6.3 Introduction to Deep Learning 31
6.4 Libraries and Tools 32
6.5 Sample Code 33-36
6.6 SQL Query 37
Chapter 6 SYSTEM TESTING 38-41
7.1 Software Testing Techniques 38
7.2 Unit Testing 38
7.3 Integration Testing 38-39
7.4 Functional Testing 39
7.5 White Box Testing 39-40
7.6 Black Box Testing 40
7.7 Test Cases 41
Chapter 8 RESULTS 42-47
Chapter 9 CONCLUSION AND FUTURE ENHANCEMENT 48-50
References 51

LIST OF FIGURES AND TABLES

Fig. No.	Figures	Page No.
5.1.1	Architecture Diagram Level 1	17
5.1.2	Architecture Diagram Level 2	18
5.3	UML Notations	20
5.4.1	Use CaseDiagram	23
5.4.2	Class Diagram	24
5.4.3	Sequence Diagram	25
5.4.4	Deployment Diagram	26
5.4.5	Activity Diagram	27
5.4.6	Component Diagram	28
5.4.7	ER Diagram	29
6.1	Working of Python Program	30
8.1	Index Page	43
8.2	Registration Page	43
8.3	Login Page	44
8.4	Home Page	45
8.5	Prediction Page	46
8.6	Result Page	47

LIST OF ABBREVATIONS

- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **DL:** Deep Learning
- **NLP:** Natural Language Processing
- **NN:** Neural Network
- **DNN**: Deep Neural Network
- **CNN:** Convolutional Neural Network
- **RNN:** Recurrent Neural Network
- **LSTM:** Long Short-Term Memory (a type of RNN)
- **GRU:** Gated Recurrent Unit (a type of RNN)
- **BERT:** Bidirectional Encoder Representations from Transformers
- **RoBERTa:** Robustly Optimized BERT Approach
- **NLI:** Natural Language Inference
- **STS:** Semantic Textual Similarity
- **TF:** TensorFlow (deep learning framework)
- **PyTorch:** Deep learning framework for model development
- **NLTK:** Natural Language Toolkit
- **spaCy:** Industrial-strength NLP library
- **API:** Application Programming Interface
- **UI:** User Interface
- **UX:** User Experience
- **CSV:** Comma-Separated Values (data file format)
- **F1-Score:** Harmonic mean of precision and recall
- **Precision:** Measure of relevant positive predictions
- **Recall:** Measure of correctly identified positives

CHAPTER 1
INTRODUCTION

1 INTRODUCTION

1.1 Motivation

With the digital world today, several terrabytes of textual information are being created in every second using social media, web sites, research papers and communication systems. This textual information has become one of the areas that is critically needed to be understood and processed. Among the most prevalent problems there is the question of two texts having the same meaning even though they are written in different ways. This difficulty is the basis of identification of paraphrases.This project is driven by the fact that it is necessary to enhance the way in which machines learn human language. Compared to human beings, machines can not naturally grasp a meaning; they follow patterns and acquired representations. The conventional rule-based methods are inadequate to deal with the diversity and complexity of natural language particularly when the sentence has a different structure and yet has the same meaning. This brings about the necessity to have smart mechanisms that have the ability to discern semantic similarity well.The other significant reason is the broad spectrum of real world uses to which paraphrase detection is applied. Indicatively, using techniques in search engines, the user can pose the same query using varied wording, and the software should be able to identify this difference to give applicable output. In chatbots and virtual assistants, it is necessary to know paraphrased user inputs to obtain the correct response. Likewise, rephrasing is a significant challenge in plagiarism detection systems to keep the originality. Question answering systems Answering the same question many times should be detected so that it can be used to enhance efficiency and prevent redundancy.Due to the rapid development of deep learning, particularly transformer-related models such as BERT and RoBERTa, there is a high chance to improve the work of paraphrase identification systems. These models have proven to have a superior performance in the development of contextual and semantic text. The reason behind this project is a need to investigate and compare these new ways with the older ones in order to learn their advantages and shortcomings.Moreover, the present project will allow acquiring practical experience in such key fields as machine learning, natural language processing, and deep learning architectures. It assists in realizing the way theoretical issues can be utilized to address practical problems. Problem-solving and analytical skills are also developed during the process of implementing, training and evaluation of various models.The other important motivation is to overcome issues of ambiguity, context preservation and structural variation in language. The project will seek to find ways of

addressing these issues by testing alternative models that can better manage them and enhance the overall accuracy of paraphrase detection.

Moreover, the project will stimulate the study of such advanced concepts as transfer learning and contrastive learning that become more significant in the present NLP applications. Knowledge of these techniques does not only enhance the performance of the model but also gives a chance to conduct research and innovation in the future.All in all, the driving force behind this project is to help in the creation of smart systems that would be more helpful at deciphering the human language, lowering the uncertainty of the communications, and making people experience better in different apps. It also seeks to establish a sound basis on the NLP techniques that can be generalized to more difficult tasks in the future.

1.2 Definition

Paraphrase Identification is a core task in Natural Language Processing (NLP) that involves determining whether two sentences convey the same meaning, even if expressed using different words or structures. As language is inherently flexible, people often express the same idea in multiple ways. For machines to effectively process and understand human language, recognizing these variations is critical. This task is particularly valuable in applications such as question-answering systems, chatbots, search engines, duplicate detection, and plagiarism checking.

This project focuses on classifying pairs of sentences into two categories: paraphrases or non-paraphrases. It approaches the problem as a supervised machine learning classification task. Initially, traditional deep learning models like Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT are implemented to establish a baseline. These models are capable of learning linguistic patterns but have certain limitations in understanding deeper semantic relations.

To address these limitations, the project proposes the use of more advanced transformer-based models such as RoBERTa. These models leverage pre-trained language representations and contrastive learning to better capture sentence semantics and contextual meaning. , for example, focuses on learning more discriminative sentence embeddings using unsupervised or supervised contrastive techniques. RoBERTa, a robustly optimized variant of BERT, is designed to enhance performance through more extensive training and fine-tuning strategies.

The project is designed for educational purposes, aiming to develop practical knowledge in NLP model design, sentence similarity detection, and the application of modern deep learning techniques. Through experimentation and evaluation on benchmark datasets, it seeks to highlight the strengths and weaknesses of each model and understand how well they perform in identifying paraphrased content. Overall, this work contributes to a deeper understanding of semantic modeling and the challenges involved in teaching machines to understand human language at a concept

CHAPTER 2
LITERATURE SURVEY

2 LITERATURE SURVEY

Related Work

1.Simple Contrastive Learning of Sentence Embeddings

Authors: Tianyu Gao, Xingcheng Yao, Danqi Chen (2021)
Link: https://arxiv.org/abs/2104.08821

introduces a novel approach for generating sentence embeddings using contrastive learning. The model is trained in an unsupervised manner by treating two augmented versions of the same sentence as a positive pair and other sentences in the batch as negatives. The supervised variant uses natural language inference (NLI) pairs. leverages BERT and RoBERTa backbones, but achieves much higher semantic similarity performance without complex architectures. Its simplicity lies in using dropout as the only augmentation, avoiding costly paraphrase generation. The model sets new benchmarks on semantic textual similarity tasks and demonstrates superior generalization. is particularly relevant for paraphrase identification because it effectively captures deep semantic meaning between sentence pairs. It has become a widely adopted method for sentence-level representation in NLP, due to its ease of training and strong performance on diverse similarity tasks.

2. Knowledge-Guided Paraphrase Identification

Authors: Haoyu Wang, Fenglong Ma, Yaqing Wang, Jing Gao (2021)
Link: https://aclanthology.org/2021.findings-emnlp.72/

This paper presents a knowledge-enhanced model for paraphrase identification by integrating external domain-specific knowledge from Wikipedia. The approach uses BM25 to retrieve relevant contextual knowledge and incorporates it into a deep learning framework through gating mechanisms. The core idea is that sentence pairs may require background knowledge to correctly classify them as paraphrases, especially in technical or medical domains. The authors evaluated their model on the PARADE and ClinicalSTS datasets, showing significant improvements over BERT-based baselines. This method is valuable in scenarios where syntactic similarity is low, but semantic similarity is high due to shared domain knowledge. The gating mechanism helps the model selectively incorporate external information, thereby

improving robustness and context sensitivity. The study emphasizes the need for knowledge-aware models in semantic similarity tasks, especially when working with domain-specific language, making it highly relevant for expanding paraphrase detection systems beyond general text.

3. ParaSCI: A Large Scientific Paraphrase Dataset for Longer Paraphrase Generation

Authors: Qingxiu Dong, Xiaojun Wan, Yue Cao (2021)
Link: https://aclanthology.org/2021.eacl-main.33/

ParaSCI introduces a large-scale dataset of scientific paraphrases, specifically targeting long and complex sentences from scientific publications. The dataset was built by extracting and aligning multiple rephrasings of scientific abstracts, resulting in over 350,000 paraphrase pairs. To better model long-form paraphrases, the authors proposed PDBERT—a specialized paraphrase detection model fine-tuned on the new dataset. This work addresses the lack of long-form paraphrase corpora in existing benchmarks, which tend to focus on short and simple sentences. The authors demonstrated that existing models struggle with length and domain complexity, and that PDBERT significantly outperforms traditional methods in this context. ParaSCI is a valuable contribution to both paraphrase detection and generation research. It opens new possibilities for applications like scientific summarization, plagiarism detection, and research content simplification. This paper is significant for anyone working on paraphrase identification in technical or formal writing.

4. Paraphrase Identification via Textual Inference

Authors: Ning Shi, Bradley Hauer, Jai Riley, Grzegorz Kondrak (2024)
Link: https://aclanthology.org/2024.starsem-1.11/

This paper explores the idea of treating paraphrase identification as a Natural Language Inference (NLI) problem. Instead of designing a standalone paraphrase model, the authors fine-tune NLI models such as RoBERTa on paraphrase tasks. Their hypothesis is that paraphrasing involves semantic entailment—if sentence A entails sentence B and vice versa, they are likely paraphrases. They tested this on multiple datasets like MRPC, QQP, and PAWS, and found

that fine-tuned NLI models outperformed traditional paraphrase classifiers. This cross-task adaptation leverages large-scale pretraining and generalization capabilities of NLI datasets like SNLI and MNLI. The paper is important because it simplifies paraphrase detection by aligning it with an existing, well-supported NLP task. This unified approach reduces training overhead and boosts performance, particularly for edge cases where paraphrase decisions are semantically subtle. It's an innovative perspective that opens up cross-domain possibilities for sentence-pair modeling.

CHAPTER 3

SYSTEM ANALYSIS

3 SYSTEM ANALYSIS

System Analysis is an essential stage of developing any software system because it is a complex examination of the current approaches, their shortcomings, and development of a better one. When applied to the context of this project, aimed at determining paraphrase identification with the help of Natural Language Processing (NLP), system analysis allows seeing how various models comprehend semantic similarity between the sentence pairs and whether they are able to cope with the issue successfully.The main task of the system analysis in the project is to assess the efficacy and the weakness of the current methods, including Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT. Although such models have also been useful in the context of classification of the text and sentence similarity, they tend to be problematic when it comes to capturing true contextual meaning and long range dependencies, as well as generalizing when dealing with dissimilar sentence structures.

This discussion reveals the limitations of traditional and baseline models such as the fact that they do not capture the intricate semantic association, they are computationally inefficient and less accurate on ambiguous and structurally diverse sentences. These constraints emphasize the necessity of more sophisticated and stronger methods.In accordance with these observations, the proposed system provides transformer-based semantic understanding models like RoBERTa and contrastive learning-based methods. These models can utilize pre-trained contextual embeddings and wide-scale training data, and thus they can capture the meaning of the sentences better and enhance the accuracy of paraphrase detection.The workflow of the project, which consists of data preprocessing, feature extraction, model training and evaluation is also defined by the system analysis. It also vividly compares the current and the suggested systems in terms of the way in which the new methodology can overcome the past shortcomings and deliver enhancements in the performance.

Comprehensively, the given stage is crucial to achieve that the created system should be efficient, scalable, and able to address the real-life problems in the field of paraphrase identification. It provides the groundwork in developing a system that would bridge the differences between the human language variability and machine comprehension.

3.1 EXISTING SYSTEM

The existing methods for paraphrase identification utilize deep learning models such as CNN, RNN, and BERT. CNN captures local syntactic patterns, while RNN handles sequential word dependencies in sentences. BERT introduces contextual word embeddings by pre-training on large corpora using a transformer-based architecture, offering improved performance over traditional methods. These models process sentence pairs and output similarity scores or classifications based on learned representations. Each of these architectures provides varying levels of semantic understanding but may struggle with deeper contextual alignment or nuanced phrasing.

3.1.1Disadvantages of existing system

CNNs have a tendency to lose long-range word relations

Convolutional Neural Networks (CNNs) are useful in capturing local structures like phrases or immediate word associations with the use of filters. They pay attention to small fragments of the sentence however. Due to this, CNNs have difficulties in comprehending relationships among words that are distant with each other in a sentence. An instance is when sentencing a long sentence or a complicated sentence, one might miss vital information in the context of the sentence therefore getting the wrong meaning.

RNNs have problems of vanishing gradient and are computationally expensive

Recurrent Neural Networks (RNNs) handle text one word at a time and it is useful in processing the order of words and the context. Nevertheless, they may encounter the vanishing gradient problem during training wherein the learning signal fades away as it passes through extensive sequences. This complicates the ability of the model to acquire long-term dependency. Also, they are slower and less efficient than parallel-processing models since RNNs handle only one word at a time, which means that they demand more computation and time.

BERT is a powerful model that is not as effective as it can be without contrastive semantic tasks

BERT is a pre-trained system that is very similar to contextual language understanding. Nevertheless, it is trained with general language tasks and might not do best with specific tasks such as paraphrase identification unless it is carefully fine tuned. Having no task-specific fine-

tuning or other methods, such as contrastive learning could make BERT unaware of the subtle differences between very similar sentences or slightly different sentences in meanings.

On small datasets, models can overfit or within paraphrases with a thin line between them.Deep learning models can overfit the data when they are trained with small datasets since they might learn patterns rather than general features. This implies that the model works well with training data but poorly on unexplored and new data. Also, paraphrase identification is not an easy task as sentences can be similar though they can be having different meanings. The subtle differences that are present might not be interpreted rightly by the models leading to erroneous classifications.

3.2 PROPOSED SYSTEM

The proposed system incorporates advanced transformer-based models— and RoBERTa—for paraphrase identification. leverages contrastive learning to generate high-quality sentence embeddings that capture deep semantic similarity, making it particularly effective for distinguishing paraphrased content. RoBERTa builds on the foundation of BERT with robust pretraining strategies and improved training data usage, offering enhanced contextual understanding. This system aims to address the limitations of earlier models by focusing on both syntactic and semantic coherence. The proposed approach includes encoding sentence pairs using these models and classifying them through similarity measures or fine-tuned classifiers. This system not only boosts classification accuracy but also improves generalization across varying sentence structures and meanings.

3.2.1. Advantages of Proposed System

Learns more semantically similar to contrastive learning

Contrastive learning allows the model to learn the meaning of sentences by learning how to discriminate between similar and dissimilar sentence pairs. The model does not simply learn the superficial words but gets to learn deeper semantic relationships. This enables it to detect paraphrases properly even in cases where the wording or sentence construction is very dissimilar.

Better performance and better generalization Compared to earlier models

Sophisticated designs including transformer-based architectures are educated with big data and are able to acquire intricate patterns of language. This leads to increased precision over the traditional models such as CNN and RNN. They also generalize more, i.e. not only on training data do they do well, but also on new, unseen pairs of sentences.

Works with complicated sentence structures and paraphrasing subtleties

The new generation models such as BERT and RoBERTa are able to elucidate the intricate structures of sentences, such as long sentences, passive voice, and other forms of grammar. They can also get finer differences and similarities in meaning, and hence this is vital in the accurate recognition of paraphrases.

With a good sentence embedding, similarity can be computed faster

Such models encode sentences into numerical formulations referred to as embeddings that encode the meaning of their sentences in a compact form. After the generation of embeddings, the similarity between the sentences becomes more efficient and quicker. This enhances the speed of the system as a whole when prediction takes place.

3.3 MODELS

The project utilizes a mix of conventional deep learning models and high-quality transformer-based architectures to perform paraphrase identification effectively with the help of sentence similarity analysis on the various levels of understanding. First, the Convolutional Neural Networks (CNN) are used to obtain the local textual features by determining the key phrases, keywords, and n-gram patterns that assist in defining the surface similarity between the sentencing pairs. In conjunction with this, Recurrent Neural Networks (RNN) are applied to sentence sequences sequentially, allowing the model to learn to comprehend the order and contextual relationship between words, vital to sentence meaning interpretation in longer sequence. To address the issues of these models with more in-depth semantic relations, the project uses Bidirectional Encoder Representations of Transformers (BERT), which is an effective pre-trained language model that processes text both forward and backward to enable the model to gain rich contextual information and enhance the understanding of sentence semantics. Continuing on this, RoBERTa (Robustly Optimized BERT Approach) is employed as an improved version of BERT and it is trained on bigger datasets with more effective strategies that lead to other improvements in terms of the accuracy, robustness, and

generalization of complex and diverse sentence structures. Besides these models, a contrastive learning-based model is also included to produce high quality sentence embeddings by learning to distinguish between similar (paraphrase) and dissimilar (non-paraphrase) sentence pairs, which further enhances the capability of the model to identify the semantic similarity even when the sentences are written in vastly different ways or even structure. Using a combination of such diverse models, the system can grasp both syntactic and semantic features of language and as a result, identify paraphrases easier and more precisely.

3.4 MODULES USED IN PROPOSED SYSTEM

3.4.1 USER

- **User Registration:** Allows new users to sign up with credentials to access the paraphrase identification system**.**
- **User Login:** Enables registered users to securely log in to use the system.
- **Prediction Page:** Provides an interface where users can input two sentences and receive classification results**.**
- **Logout:** Lets users safely exit the system, ensuring their session and data are securely closed**.**

3.4.2 SYSTEM

- **Data Import and Preprocessing:** Loads and cleans the sentence-pair dataset, performs tokenization, lowercasing, and removes noise to prepare it for model training.
- **Training Module:** Trains the selected deep learning models (CNN, RNN, BERT, RoBERTa) using the preprocessed sentence pairs.
- **Prediction Module:** Takes new sentence pairs as input and predicts whether they are paraphrases or not based on the trained model.
- **Backend Module**: Manages communication between the user interface and model logic, handling tasks like input validation, data flow, and prediction output.

3.5 ALGORITHMS USED

Convolutional Neural Network (CNN)

CNNs are primarily used in image processing but are effective in text classification by capturing local features. For sentences, CNN applies convolutional filters over word embeddings to detect important phrases or n-grams, enabling the model to learn spatial patterns in the sentence pair.

Recurrent Neural Network (RNN)

RNNs process sequential data by maintaining hidden states, making them suitable for modeling word order and context in sentences. Variants like LSTM or GRU help the model remember long-range dependencies between words, which is important for understanding sentence meaning.

Bidirectional Encoder Representations from Transformers (BERT)

BERT is a powerful transformer-based language model pre-trained on large corpora to learn contextual word representations. It considers both left and right context simultaneously, improving understanding of sentence semantics. For paraphrase identification, BERT encodes sentence pairs jointly to capture their relationships.

Robustly Optimized BERT Approach (RoBERTa)

RoBERTa is an optimized variant of BERT that trains longer and on more data with dynamic masking strategies. This results in improved generalization and robustness, making it highly effective for sentence-pair classification tasks like paraphrase identification.

CHAPTER 4

SYSTEM REQUIREMENTS SPECIFICATION

4. SYSTEM REQUIREMENTS SPECIFICATION

Software requirements specifications (SRS), also known as software system requirements specifications, offer a comprehensive description of the duties that a system must do. The use cases in this section describe how the software interacts with its users. The SRS also contains non-functional specifications in addition to the usage case. Non-functional specifications are criteria that limit design or execution (such as requirements for performance engineering, quality standards or design constraints).

4.1 SOFTWARE REQUIREMENTS

Operating System	: Windows 7/8/10
Programming Language	: Python
Libraries	: Pandas, Numpy, scikit-learn.
IDE/Workbench	: Visual Studio Code
DataSet	: BenchMark

4.2 HARDWARE REQUIREMENTS

Processor	- I3/Intel Processor
Hard Disk	- 160GB
Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- SVGA
RAM	- 8GB

4.3 FEASIBILITY STUDY

A feasibility study is a critical process in the development of a system that determines the practicality, whether a proposed project makes sense or not, and whether the project is viable or not. It assists in establishing whether the system is possible to be developed using the resources available, time and technology and delivering the desired objectives. In this project, where the paraphrase identification is conducted based on the Natural Language Processing (NLP) and deep learning models, the feasibility study is conducted in three areas, namely,

economic, technical, and behavioural ones, in order to make sure that the system implementation is successful.

The proposed system uses machine learning methods and pre-trained models: CNN, RNN, BERT, and RoBERTa to categorize a pair of sentences by a paraphrase or non-paraphrase. Because this project is rather academic and research-based, one should examine whether the tools and the infrastructure needed, as well as the user acceptance can be found in accordance with the system requirements. The viability study ensures that the project is a viable one and can be executed successfully using the available resources and expertise.

4.3.1 Economic Feasibility

Economic feasibility is used to determine the economics of the project by incorporating the cost of the project against the projected gains. The cost of development is low in this project since the project will use open-source tools, libraries, and frameworks, including Python, Pandas, NumPy, and machine learning libraries. The development environment, for example, Visual Studio Code, is also free and this reduces software costs.

The hardware is simple, namely a typical system with average processing power and memory that is readily available. The project is cost effective as it does not require any costly infrastructure or customized equipment. Moreover, the pre-trained models such as BERT and RoBERTa are made publicly available and thus the cost of training models is also minimized.

4.3.2 Technical Feasibility

Technical feasibility evaluates the availability of the necessary technology, tools and expertise to develop the system successfully. The proposed project is technically viable because advanced machine learning frameworks and libraries can be used to support NLP tasks. It is written in Python, a popular data science and machine learning language. PyTorch, scikit-learn, and TensorFlow are libraries that have robust support in the construction and training of deep learning models. Data preprocessing, tokenization, model training, and evaluation are also standard processes in this project that are highly covered by the existing tools and documentation. The calculations involved are not too complex and the system can be built on a regular computer without any advanced hardware.

4.3.3 Behavioural Feasibility

Behavioural feasibility puts attention on how acceptably and usable the system is by the users. It determines how user-friendly the system would be and the level of readiness of the users to embrace it. In the given project, the system is developed to have simple and easy-to-use interface where the user can enter 2 sentences and the system will then give a prediction to the user whether the sentences are paraphrasing. The operation is easy and needs little user input and no technical skills to do it. The system offers prompt and precise results, and this increases user satisfaction and will motivate the use. Moreover, the fact that the application is used to solve common issues like finding similarities in content and sentence similarity means that the application would be useful and relevant to users.

No major behavioral obstacles exist because the system does not compel the users to alter their current habits and workflows. It is meant to support the users and not to make their jobs difficult. As such, the system is easy to use, acceptable and easy to behave.

4.4 FUNCTION AND NON-FUNCTIONAL REQUIREMENTS

Requirement's analysis is very critical process that enables the success of a system or software project to be assessed. Requirements are generally split into two types: Functional and non-functional requirements.

4.4.1 Functional Requirements:

These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

Examples of functional requirements:

1) Authentication of user whenever he/she logs into the system
2) System shutdown in Solar prediction.
3) A verification email is sent to user whenever he/she register for the first time on some software system.

4.4.2 Non-functional requirements:

These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements. They basically deal with issues like:

- Portability
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Reusability
- Flexibility

Examples of non-functional requirements:

1) Emails should be sent with a latency of no greater than 12 hours from such an activity.
2) The processing of each request should be done within 10 seconds
3) The site should load in 3 seconds whenever of simultaneous users are > 10000

CHAPTER 5
SYSTEM DESIGN

5. SYSTEM DESIGN

5.1 ARCHITECTURE DESIGN

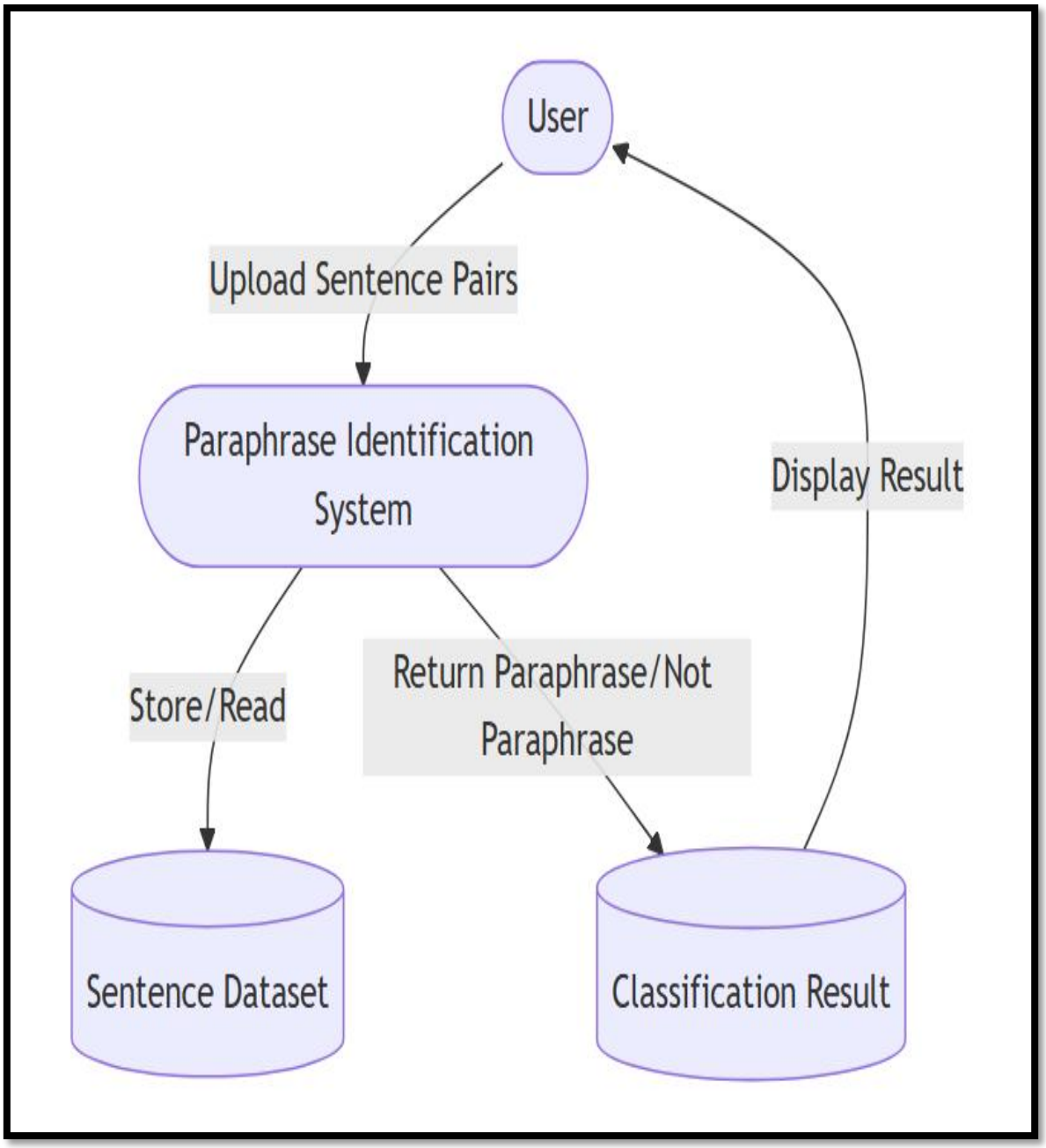


Fig-5.1.1: System Architecture Diagram (Level 1)

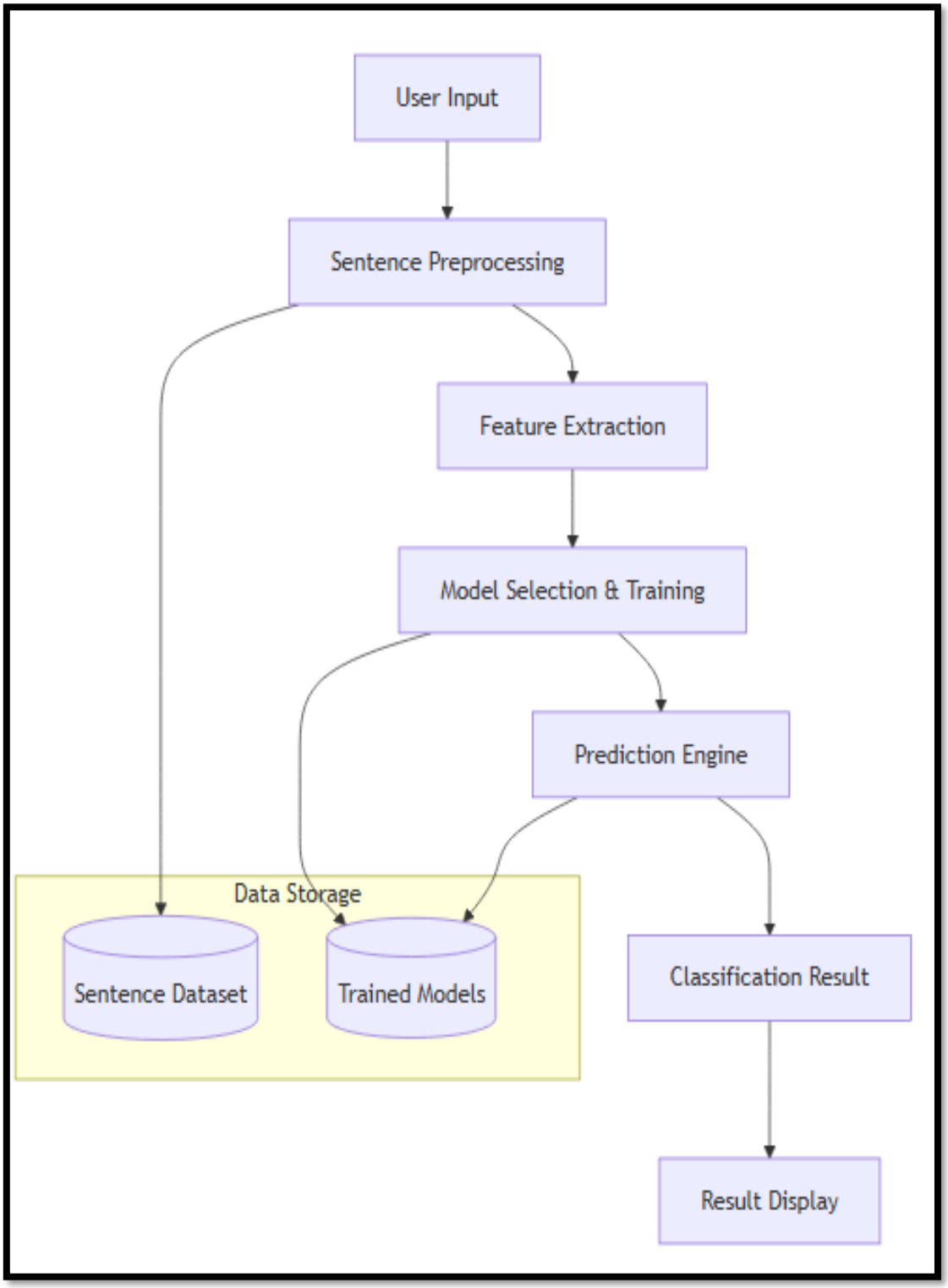


Fig-5.1.2: System Architecture Diagram (Level 2)

Input and output design, are considered as the core elements of the overall design of the Paraphrase Identification System in that these two factors directly affect the efficiency, accuracy, and usability of the system. Input is described as raw data in which the user feeds the system, which is normally in the form of sentence pairs, which undergo processing through the system to establish whether they are paraphrases or otherwise. An effective input mechanism will provide data that is collected in a system, precise and convenient to use. It is concerned with simplification of input, elimination of redundancy and the addition of validation processes to limit mistakes made during information input. Under the input design, the manner in which the users interrelate with the system is also taken into account, making the interface simple, clear and consistent so that the user can easily make the necessary information without confusion. Design of inputs will enhance the quality of the data and will also increase the overall system performance by making sure that only the relevant and properly formated data is processed.

Output design on the other hand is also vital since it determines how the information that is processed should be displayed to the user. Outputs of the system take the shape of results of the classification, which is the indication of the presence of paraphrases and non-paraphrases between the sentences provided. The design of the output is well structured and thus the final presentation is precise, understandable, and clear to make the information comprehendible. It is concerned with giving relevant information only in a proper format without any unnecessary information that can bewilder the user. Also, the output must be provided in a timely fashion that will facilitate fast decision-making and increase user satisfaction. The design also makes the results to be constantly displayed and available to the targeted users. In general, a well-integrated input and output design will help to have a seamless flow of data in the system, enhance user experience, enhance reliability, and make sure that the system can fulfill its intended purpose effectively and correctly.

5.2 INTRODUCTION TO UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artefacts of software system, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

The Internet has also made many structural problems worse while simplifying some tasks. The Unified Modeling Language (UML) was created to satisfy these requirements. Simply described, systems design is the process of creating a system's architecture, components, modules, interfaces, and data to meet certain goals. This can be accomplished fast using UML diagrams. Throughout the project, eight fundamental UML diagrams were explained.

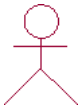
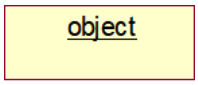
- ➢ Use Case Diagram
- ➢ Class Diagram
- ➢ Activity Diagram
- ➢ Sequence Diagram
- ➢ Collaboration Diagram
- ➢ State chart Diagram
- ➢ Component Diagram
- ➢ Deployment Diagram

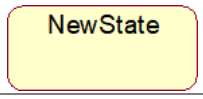
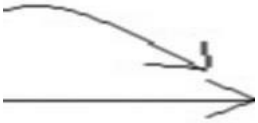
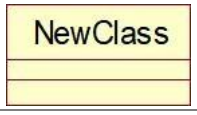
5.2.1 GOALS

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modelling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

5.3 UML NOTATIONS

S.NO	SYMBOL NAME	NOTATION	DESCRIPTION
1.	Initial Activity		This diagram depicts the flows initial point or activity.
2.	Final Activity		A bull's eye icon marks the conclusion of the activity graphic.
3.	Activity		Represented by a rectangle with a rounded edge.
4.	Decision		One that requires decision-making.
5.	Use Case		Explain how a user and a system communicate.
6.	Actor		A function a user has in relation to the system.
7.	Object		A Real-Time entity.

8.	Message		To communicate between the lives of object.
9.	State		It depicts events that occur during an objects lifetime.
10.	Initial State		Represents the objects initial state.
11.	Final State		Represents the objects final state.
12.	Transition		Label the transition with the event that triggered it and the action that result from it .
13.	Class		A group of items with similar structures and behaviours.
14.	Class		Relationship between classes.
15.	Generalization		Relationship between more general class and a more specific class.

5.4 UML DIAGRAMS

5.4.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

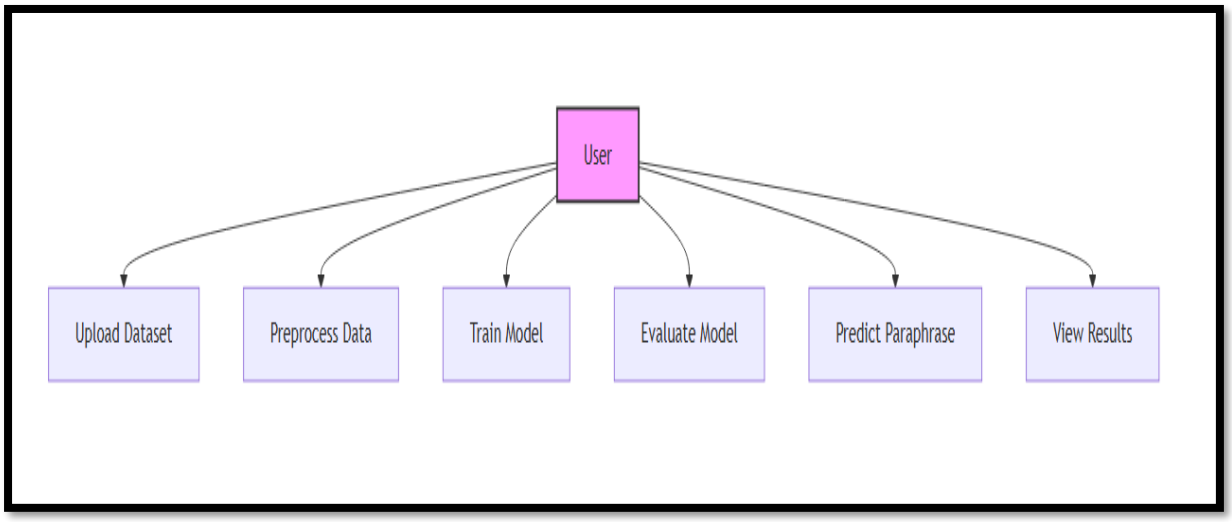


Fig 5.4.1 : Use Case Diagram

5.4.2 CLASS DIAGRAM

A class diagram in the Unified Modeling Language (UML) is a type of static structural diagram used in software engineering to show the classes, properties, and relationships between the classes that make up a system. It is utilised in analysis to show the system's specifics. Architecture examines the class diagram to determine which classes have an excessive number of functions and, if any do, whether they should be divided. The connections between the classes are made. The Class Diagram is a tool used by developers to create classes. A class diagram is a group of related objects that are all connected and have the same features, operations, relationships, and connections and regulations referred to as semantics. A class is a huge group of items in a production.

In the Unified Modeling language, a class diagram is a type of static structural diagram that

displays the functions, interactions, and relationships between objects to depict a system's structure. The cornerstone of object-oriented modeling is the class diagram. Image, build dataset, pre-processing, segmentation, and classification are the classes represented in together with the corresponding properties, processes, and relationships between those classes.

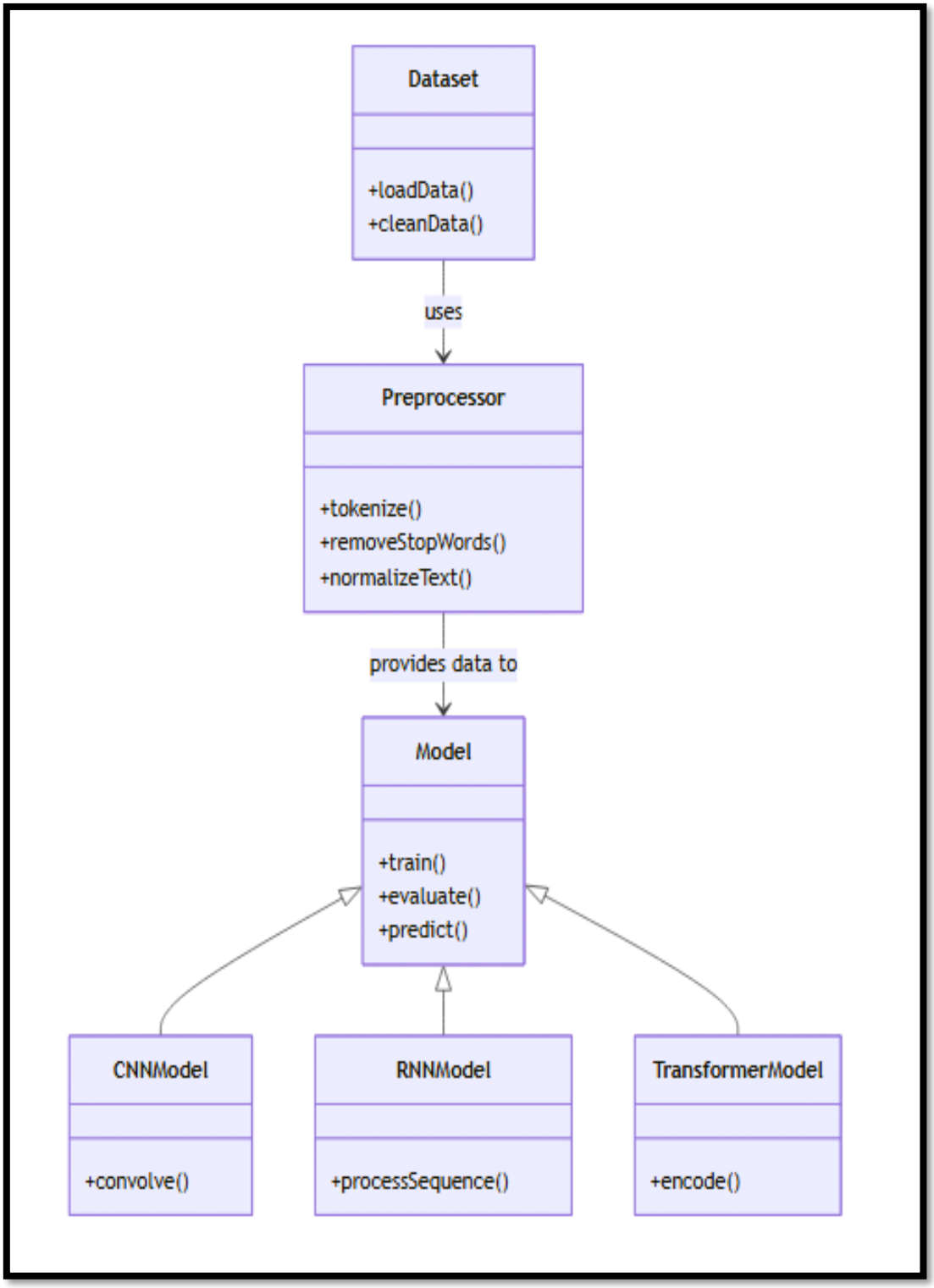


Fig 5.4.2: Class Diagram

4.3 SEQUENCE DIAGRAM

In the Unified Modelling Language (UML), a sequence diagram is a type of interaction diagram that illustrates the order and relationship between activities. A message sequence chart is the name given to it. Sequence diagrams include timing diagrams, event-trace diagrams, and representations of event contexts. One can also refer to a sequence diagram as an event diagram or an event scenario. Sequence diagrams show how a system's components interact with one another. The requirements for both new and current systems are frequently described and understood by entrepreneurs and software engineers using these diagrams. An interaction diagram that emphasises the timing of message delivery. Depending on their lifespan and the messages they transmit or arrange over time, objects taking part in an interaction are represented in a sequence diagram.

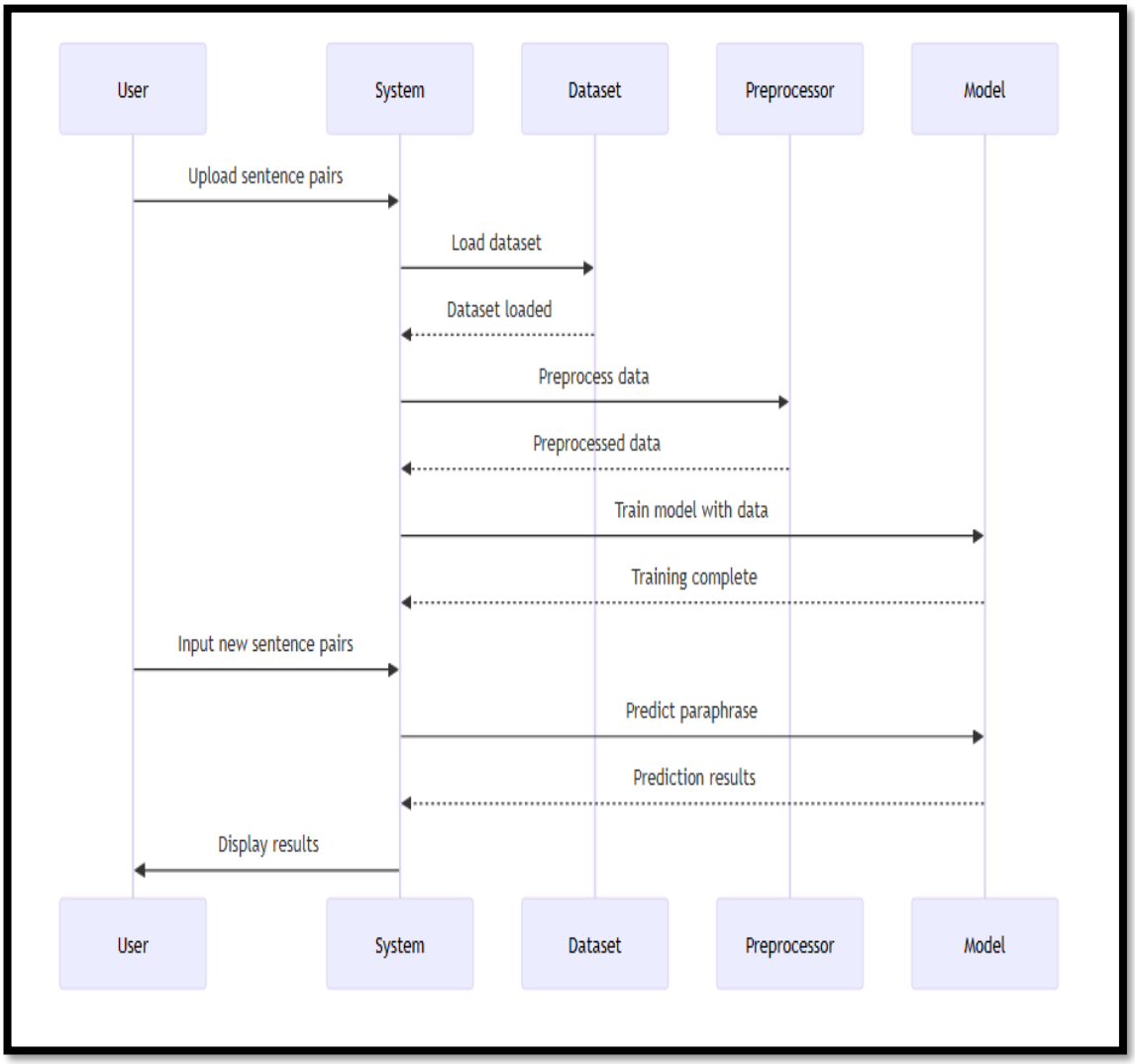


Fig 5.4.3: Sequence Diagram

5.4.4 DEPLOYMENT DIAGRAM

The hardware and software components that make up a deployment are described using deployment diagrams. Diagrams of components and deployments have a lot in common. Diagrams of the components' deployment in hardware are shown in deployment diagrams, which are used to describe the components. The software artefacts of a system are the primary emphasis of UML. However, these two particular diagrams are meant to highlight the hardware and software parts. In contrast to deployment diagrams, which are designed to concentrate on a system's hardware topology, most UML diagrams are used to manage logical components. The system engineers utilize diagrams for deployment. You can characterize the function of deployment diagrams as: • Think about how a system's hardware is organized. • Explain the hardware elements that are deployed in order to run software components. • Tell us about the runtime processing nodes.

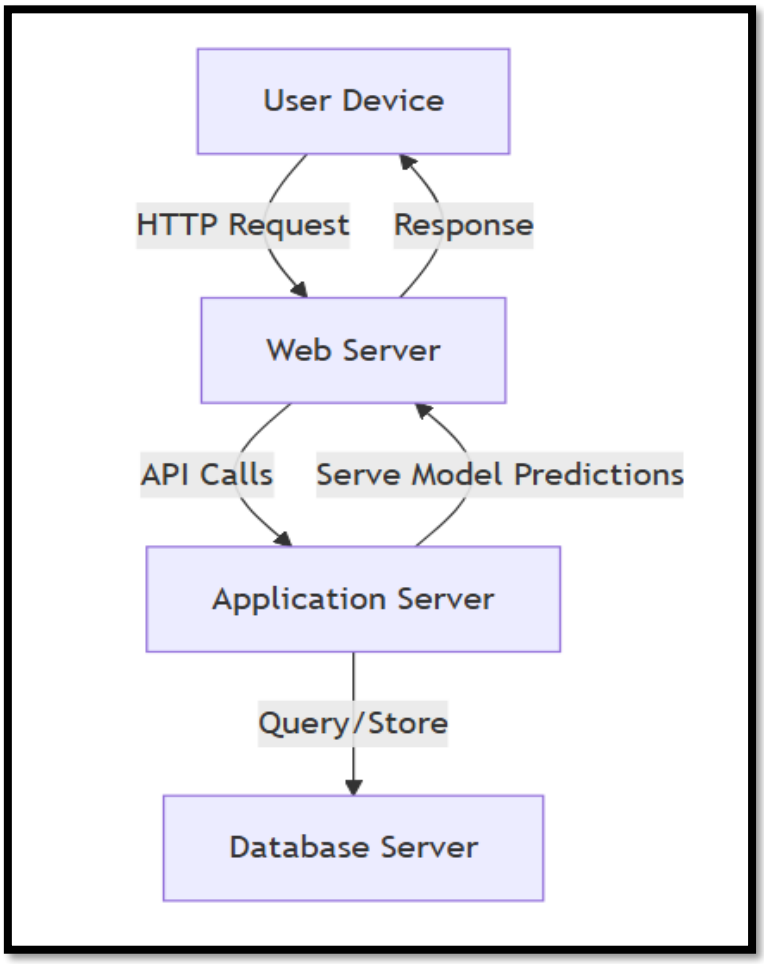


Fig 5.4.4 Deployment Diagram

5.4.5 ACTIVITY DIAGRAM:

Activity diagrams offer choice, iteration, and concurrency in their depiction of the work flows of evolving tasks and actions. The operational and business processes of system components can be represented in detail using activity flowcharts in the Unified Modified Language. An activity diagram illustrates the entire control flow. A flowchart with specific states is similar to an activity diagram. With the activity diagram, you can keep track of the sequence of actions occurring in your system. Activities look like states; however, they are a little more rounded. They are stateless because they take place and then go unabatedly to the following state. The "diamond" conditional branch determines which activity to switch to based on a characteristic and is also stateless. Activity Diagram includes

• Action states.

• Transition.

• Objects.

• Contains Fork, Join and branching relations along with flow Chart symbols.

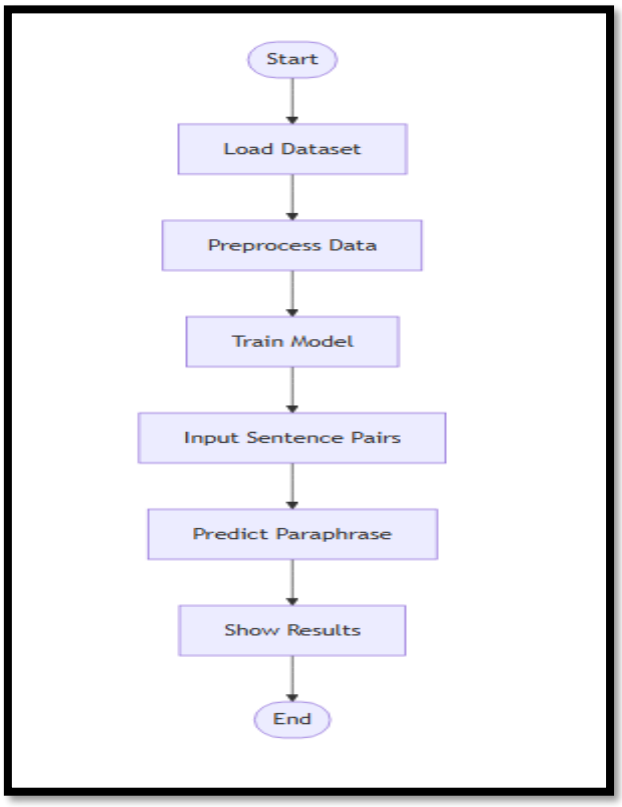


Fig 5.4.5 : Activity Diagram

5.4.6 COMPONENT DIAGRAM:

A specific type of diagram in UML is called a component diagram. The goal is also distinct from the previous diagrams mentioned. Although it defines the components utilized to provide certain functionalities, it does not describe the system's functionality as a whole. Component diagrams are used to represent the physical parts of a system from that perspective. These parts include files, libraries, and other things. A static implementation view of a system is another way to explain component diagrams. The arrangement of the components at a specific time is represented by static implementation. The entire system cannot be represented by a single component diagram; instead, a collection of diagrams is employed. The component diagram's goal can be summed up as follows: 1. Identify the parts of a system visually. 2. Use both forward and reverse engineering to create executables. 3. Explain how the components are arranged and their connections.

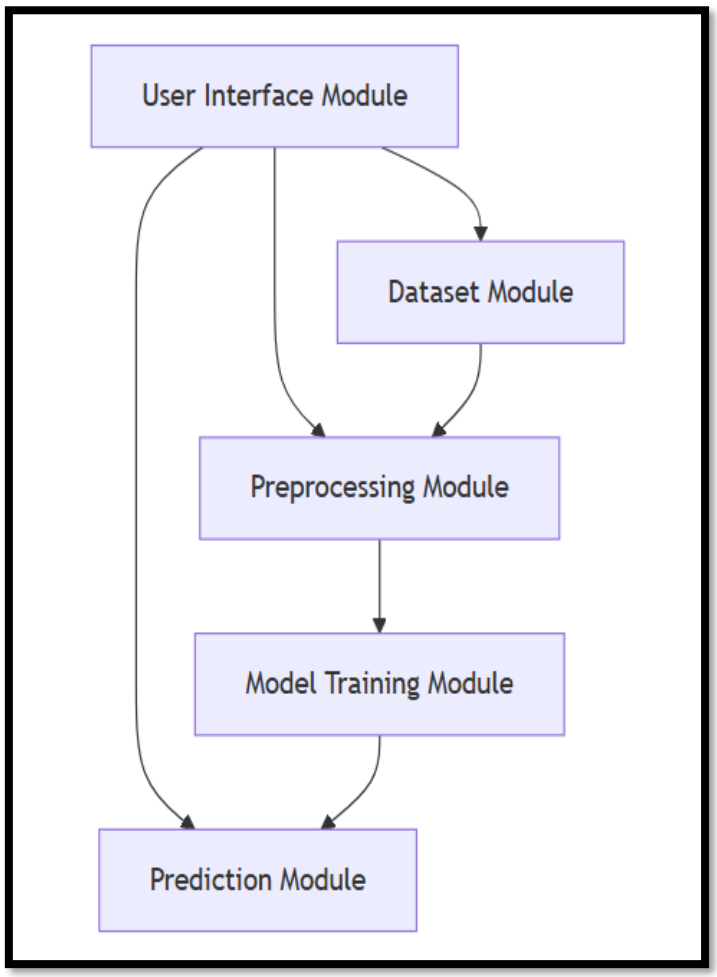


Fig 5.4.6: Component Diagram

5.4.7 ER DIAGRAM:

An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram). An ER model is a design or blueprint of a database that can later be implemented as a database. The main components of E-R model are: entity set and relationship set.

An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and their attributes, ER diagram shows the complete logical structure of a database. Let's have a look at a simple ER diagram to understand this concept.

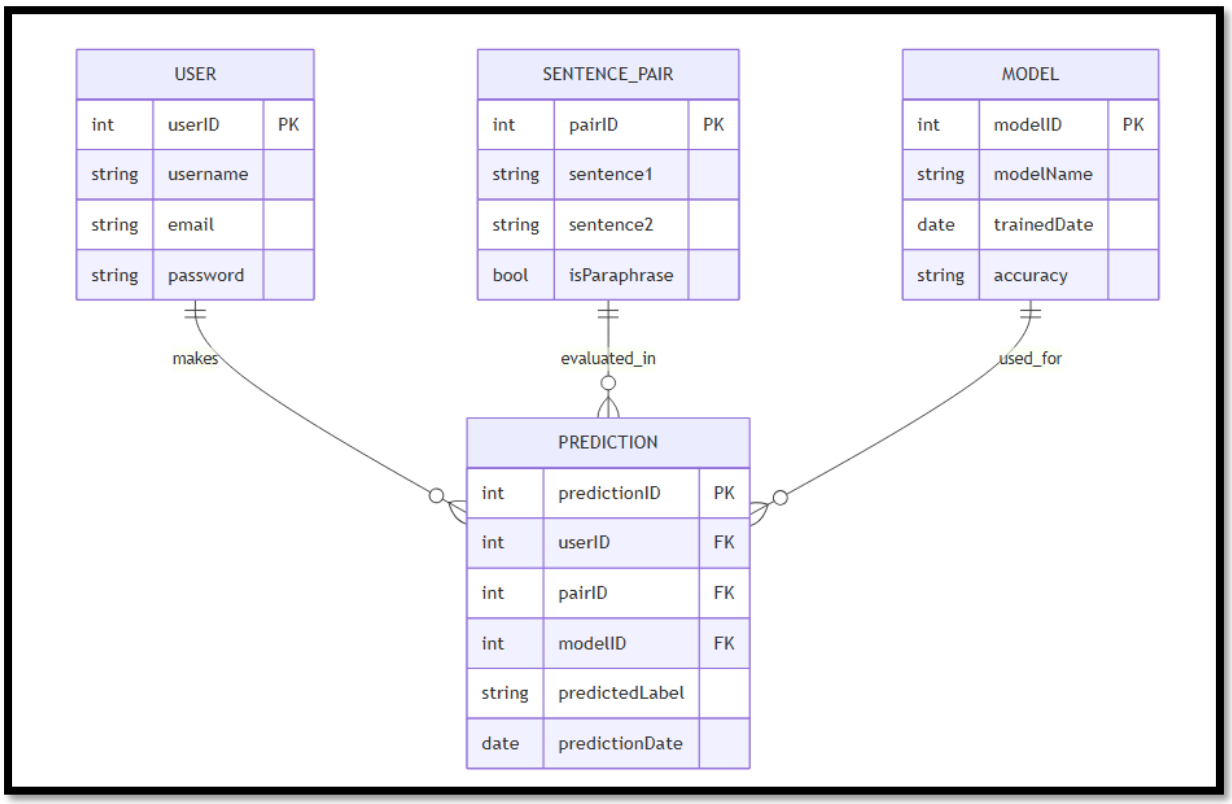


Fig 5.4.7 : ER Diagram

CHAPTER 6

SYSTEM CODING AND IMPLEMENTATION

6. SYSTEM CODING AND IMPLEMENTATION

6.1 Introduction to python programming language:

Popular computer language Python is renowned for its simple syntax, scalability, and application in artificial intelligence (AI) and machine learning. Python is a key component of the systems used by some of the biggest companies in the world, including Google, NASA, and Facebook. Python is an object-oriented programming language. By definition, it is a high-level programming language, enabling the construction of both simple and complex processes.

Features:

• Simple

• Easy

• Portable• Object oriented

• High Level

• Open Source and Free

• Support for GUI

• Interpreted

• Dynamic

• Readable

• Extendable

• Scalable

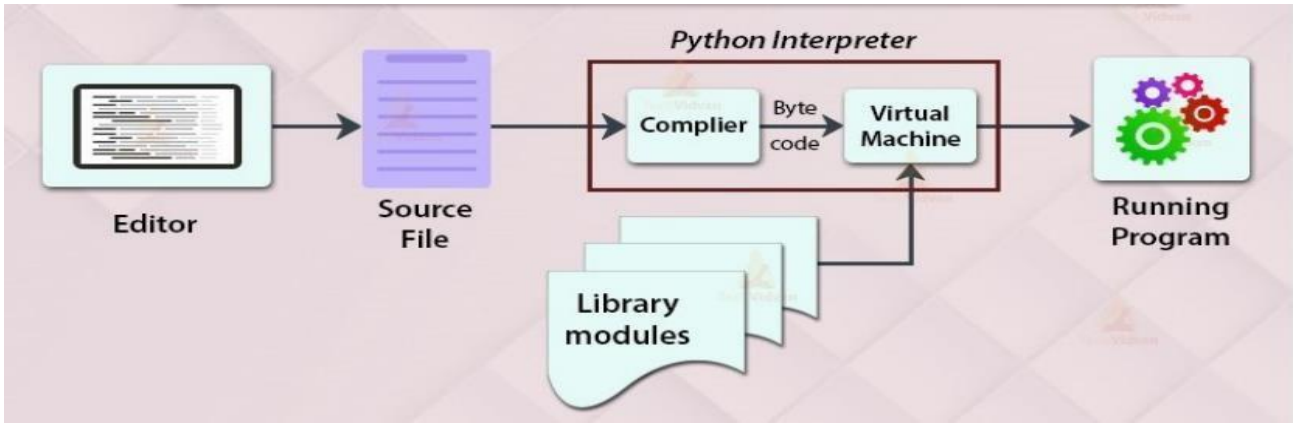


Fig 6.1: Working of Python Programming

6.2 Benefits of Python

- Python may be used to create prototypes, and because it is so simple to use and read, it can be done rapidly.
- The majority of platforms for automation, data mining, and big data rely on Python.
- Compared to large languages like C# and Java, Python offers a more productive coding environment. By using Python, seasoned programmers tend to stay more organised and productive.
- Even if you're not an experienced programmer, Python is simple to read. Everyone can start using the language; all it needs is some perseverance and lots of practise.
- Additionally, this makes it a perfect choice for use by large development teams and teams with multiple programmers.
- Django is a full and open-source web application framework that is powered by Python. The process of developing software can be made simpler by using frameworks like Ruby on Rails.
- Because it was built by the community and is open source, it has a huge fan base. Millions of like-minded programmers use the language every day and keep its foundational features up to date. As time goes on, Python's most recent version continues to get updates and improvements. This is a fantastic method of connecting with other developers.

6.3 Introduction to Deep Learning

Deep Learning is a branch of Machine Learning that deals with the application of multiple-layer artificial neural networks to characterize and comprehend complicated patterns in data. It has proved to be an effective method of problem solving concerning Natural Language Processing (NLP), like text classification, sentiment analysis and paraphrase recognition. In contrast to conventional machine learning systems, deep learning systems can automatically extract the feature representations of raw data, thereby alleviating the necessity to perform feature engineering. Deep learning is important in this project as it is used to find semantic similarities between pairs of sentences. Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are models applied to learn syntactic and sequence patterns in the text, whereas more advanced models based on transformers, including BERT and RoBERTa, are utilized to learn the contextual meaning. These models use large-size pre-trained embeddings and fine-tuning methods to have a high accuracy of paraphrase detection tasks. The system is capable of dealing with differences in sentence structure, ambiguity, and context sensitivity due to deep learning, which makes it applicable to NLP tasks in the real world.

6.4 Libraries and Tools

This project is implemented with the help of Python because of its simplicity, flexibility, and a high capacity to support machine learning and NLP applications. Data processing, model building and evaluation are performed by using various libraries.

NumPy:

It is applied in numerical calculations and in manipulation of multi-dimensional arrays. Most machine learning activities are based on it.

Pandas:

It is used to manipulate data and preprocess it. It assists in loading datasets as well as data cleaning and pairing of sentences to be used in training and testing.

Scikit-learn:

It offers necessary functionality to perform preprocessing, model evaluation, and metrics of performance, including accuracy, precision, recall, and F1-score.

TensorFlow / PyTorch:

Apache is used in the creation and training of deep learning frameworks. These frameworks present the effective mechanisms of developing the neural networks like CNNs and RNNs.

Transformers (Hugging Face Library):

Transformers can be used to implement pre-trained models, such as BERT and RoBERTa. The library eases the tasks of loading, fine-tuning and applying transformer models in tackling NLP tasks.

NLTK (Natural Language Toolkit):

Can be utilized in simple NLP preprocessing like tokenization, stop-word removal and cleanup of text.

spaCy (mandatory):

Designed to perform complex NLP functionality like linguistic processing and effective processing of text.

Matplotlib / Seaborn (in case of utilization):

It is applied to visualize the results and model performance, as well as to evaluate metrics.

6.5 Code

```
from flask import Flask,render_template,flash,redirect,request,send_from_directory,url_for,
send_file
import mysql.connector, os
from flask import Flask, request, render_template, jsonify
import torch
from transformers import BertTokenizer, BertForSequenceClassification
app = Flask(__name__)

mydb = mysql.connector.connect(
  host="localhost",
  user="root",
  password="",
  port="3306",
  database='paraphrase'
)

mycursor = mydb.cursor()

def executionquery(query,values):
  mycursor.execute(query,values)
  mydb.commit()
  return

def retrivequery1(query,values):
  mycursor.execute(query,values)
  data = mycursor.fetchall()
  return data

def retrivequery2(query):
  mycursor.execute(query)
  data = mycursor.fetchall()
  return data
```

```
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        name = request.form['name']
        email = request.form['email']
        password = request.form['password']
        c_password = request.form['c_password']
        if password == c_password:
            query = "SELECT UPPER(email) FROM users"
            email_data = retrivequery2(query)
            email_data_list = []
            for i in email_data:
                email_data_list.append(i[0])
            if email.upper() not in email_data_list:
                query = "INSERT INTO users (name, email, password) VALUES (%s, %s, %s)"
                values = (name, email, password)
                executionquery(query, values)
                return render_template('login.html', message="Successfully Registered!")
            return render_template('register.html', message="This email ID is already exists!")
        return render_template('register.html', message="Confirm password is not match!")
    return render_template('register.html')

@app.route('/login', methods=["GET", "POST"])
def login():
    if request.method == "POST":
```

```
        email = request.form['email']
        password = request.form['password']

        query = "SELECT UPPER(email) FROM users"
        email_data = retrivequery2(query)
        email_data_list = []
        for i in email_data:
            email_data_list.append(i[0])

        if email.upper() in email_data_list:
            query = "SELECT UPPER(password) FROM users WHERE email = %s"
            values = (email,)
            password__data = retrivequery1(query, values)
            if password.upper() == password__data[0][0]:
                global user_email
                user_email = email

                return redirect("/home")
            return render_template('login.html', message= "Invalid Password!!")
        return render_template('login.html', message= "This email ID does not exist!")
    return render_template('login.html')

@app.route('/home')
def home():
    return render_template('home.html')

from transformers import BertTokenizer, BertForSequenceClassification

# Load model and tokenizer once when starting the app
model = BertForSequenceClassification.from_pretrained("Paws_Bert_model")
tokenizer = BertTokenizer.from_pretrained("Paws_Bert_tokenizer")
model.eval()
```

```
def predict_paraphrase(text1, text2):
    inputs = tokenizer(text1, text2, return_tensors='pt', padding=True, truncation=True)
    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.softmax(outputs.logits, dim=1)
        predicted_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0][predicted_class].item()
    return predicted_class, confidence

@app.route('/analys', methods=["GET", "POST"])
def analys():
    prediction = None
    confidence = None

    if request.method == "POST":
        text1 = request.form.get("text1", "")
        text2 = request.form.get("text2", "")
        if text1 and text2:
            predicted_class, confidence = predict_paraphrase(text1, text2)
            label = "paraphrase" if predicted_class == 1 else "not a paraphrase"
            prediction = f"Prediction: {label} (confidence: {confidence:.2f})"
        else:
            prediction = "Please provide both texts."

    return render_template('analys.html', prediction=prediction)

if __name__ == '__main__':
    app.run(debug = True)
```

6.6 SQL QUERY

```
DROP DATABASE IF EXISTS paraphrase;
CREATE DATABASE paraphrase;
USE paraphrase;

CREATE TABLE users (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(225),
   email VARCHAR(225),
   password VARCHAR(225)
);
```

CHAPTER-7
SYSTEM TESTING

7 SYSTEM TESTING

7.1 SOFTWARE TESTING TECHNIQUES

Software testing is a method for evaluating the quality of software products and identifying defects so that they can be rectified. Software testing makes an effort to accomplish its goals, but there are significant constraints. On the other side, for testing to be effective, dedication to the set objectives is required.

7.1.1 Testing Objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

7.1.2 Test Case Design

Every engineering product can be tested in one of these.

7.2 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

7.3 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were

individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

7.4 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.5 WHITE BOX TESTING

White Box testing, also known as structural testing, clear box testing, open box testing, and transparent box testing, is covered in this article. It focuses on evaluating the infrastructure and software's fundamental code against current inputs and anticipated and desired outcomes. It emphasises internal structure analysis and is focused on a program's internal activities. To construct test cases for this type of testing, programming knowledge is needed. Focusing on the inputs and outputs of the software while also ensuring its security is the core aim of white box testing. The phrases "clear box," "white box," and "transparent box" all allude to being able to see through the exterior covering of the software. White testing a box is used by designers. This stage involves testing every line of the program's code. Prior to handing off the

programme or software to the testing team, the developers run white-box testing on it to ensure that it conforms with the requirements and to identify any mistakes.

Before releasing the project to the testing team, the developer fixes the issues and does one round of white box testing. In this case, fixing problems includes removing the problem and activating the specific functionality of the application. For the following reasons, the test engineers won't be helping to fix the problems: Resolving the problem might impair other features. As a result, developers should keep making advancements while the test engineer should constantly look for faults. If the test engineers spend most of their time fixing problems, they might not be able to find any new flaws in the program.

White box testing is essential for uncovering vulnerabilities and ensuring that the code meets the required standards of performance and security. By examining the internal workings of the software, developers can detect logical errors, potential security breaches, and other critical issues that might not be apparent through external testing alone. This method is particularly effective in optimizing code efficiency and robustness, as it allows developers to scrutinize individual paths and conditions within the code. Through meticulous inspection, developers can verify that each module or component functions correctly and integrates seamlessly with the rest of the system. Furthermore, white box testing promotes better coding practices by enforcing a high level of scrutiny and discipline. Developers are encouraged to write cleaner, more maintainable code, knowing that every line will be examined for potential flaws. This leads to improved software quality and a reduction in long-term maintenance costs

7.6 BLACK BOX TESTING

Testing software applications' functionalities without having access to the internal code structure, implementation details, or internal paths is known as "black box testing" in the software industry. The term "black box testing" refers to a sort of software testing that is solely concerned with the input and output of software program as well as the requirements and specifications for software. You are free to use any software package you choose as a Black-Box. A few examples include an Oracle database, a Google website, the Windows operating system, or even your own custom program. You can test these applications using black box testing by focusing just on their inputs and outputs and ignoring any awareness of how their underlying code is implemented. Black box testing is crucial in the software development process because it simulates real-world scenarios that end-users might encounter.

By focusing on the functionality rather than the internal mechanics, testers can identify how well the software performs under various conditions and whether it meets user expectations. This type of testing helps to validate that the software operates correctly from the user's

perspective, ensuring that all features and functions behave as intended. It is especially useful for uncovering issues related to usability, performance, and compliance with requirements. Moreover, black box testing is versatile and can be applied at different levels of software development, from individual units to entire systems. It allows testers to work independently of the development team, which can provide an unbiased assessment of the software's quality. Since the testers do not need to understand the code, black box testing can be performed by individuals with different skill sets, including those without programming expertise. This inclusivity broadens the scope of testing, allowing for a more diverse range of test cases and scenarios to be evaluated. By concentrating on the user experience and external behaviors, black box testing complements other testing methods, contributing to the overall robustness and reliability of the software.

7.7 TEST CASES

Input	Output	Result
Input Dataset	Providing Application	Success

Test Cases Model Building

S.NO	Test Cases	Input	Expected Output	Actual Output	P/F
1	Read the dataset	Dataset file path	Dataset should be loaded successfully	Dataset loaded successfully	P
2	Data loading	Dataset file	Data loading should be performed without errors	Data loaded successfully	P
3	Data preprocessing	Raw sentence pairs	Cleaned and tokenized sentences	Data preprocessing completed successfully	P
4	Model building	Preprocessed data	Model should be trained using selected algorithms	Model created and trained successfully	P
5	Prediction	New sentence pairs	Model should classify pairs as paraphrase/non-paraphrase	Predictions generated successfully	P

CHAPTER 8
RESULTS

8 RESULTS

OUTPUT SCREENS



FIG 8.1 : INDEX PAGE

A paraphrase Identification System entry point is the index page. It offers a simple and easy to use interface with a clear title and menu options like Home, About, Register and Login. The page clearly shows the intention of the system using a bold title of AI-Powered Paraphrase Detection. It also has a short tagline that describes the application of deep learning and transformer models. There are 2 key action buttons: Register and Login. The background design is more appealing to the eye and makes it easier to use. This page is easy to navigate and serves as the entry point to any user. It has been made easy and user-friendly.

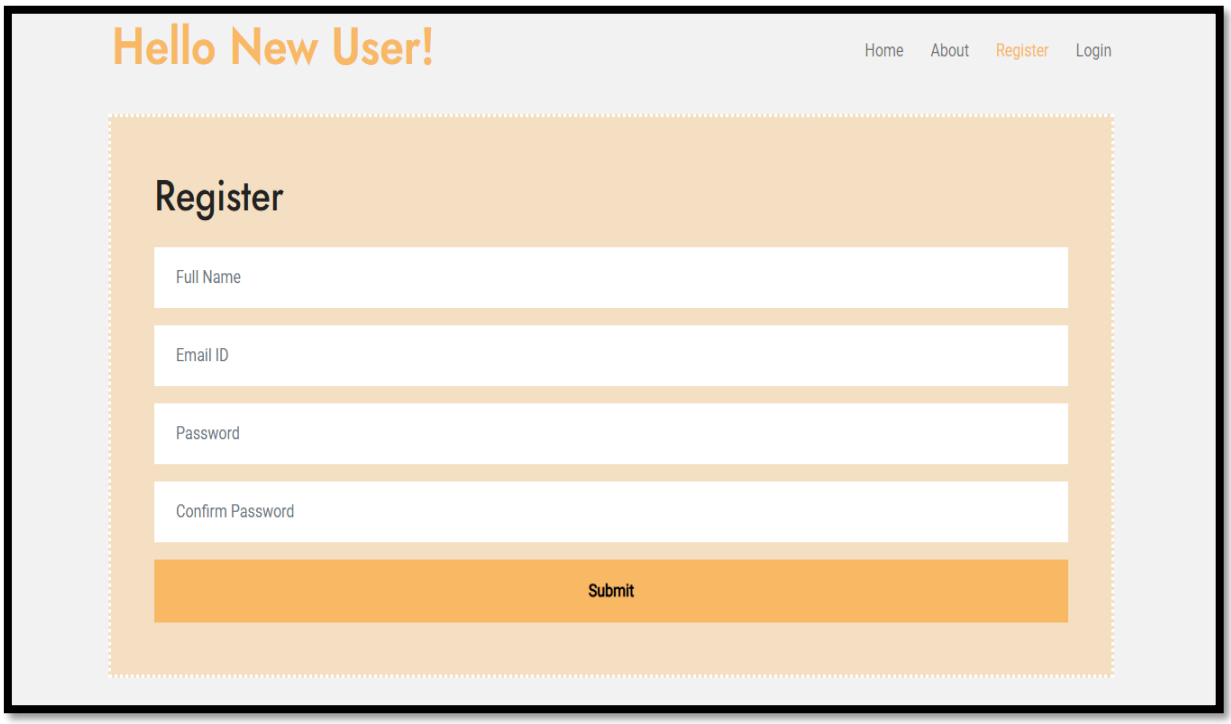


FIG 8.2: REGISTRATION PAGE

New users are able to register in the system through the registration page. It has user details like username, email and password input fields. Correct data entry and error prevention is done through proper validation mechanisms. The page will make sure that only authenticated users access the system features. Its design is easy to use and simple to comprehend, hence is open to any user. It is after the successful completion of the registration process that a user is then allowed to log in. This module improves the security of the system and management of the users. It is also useful in keeping user specific records.

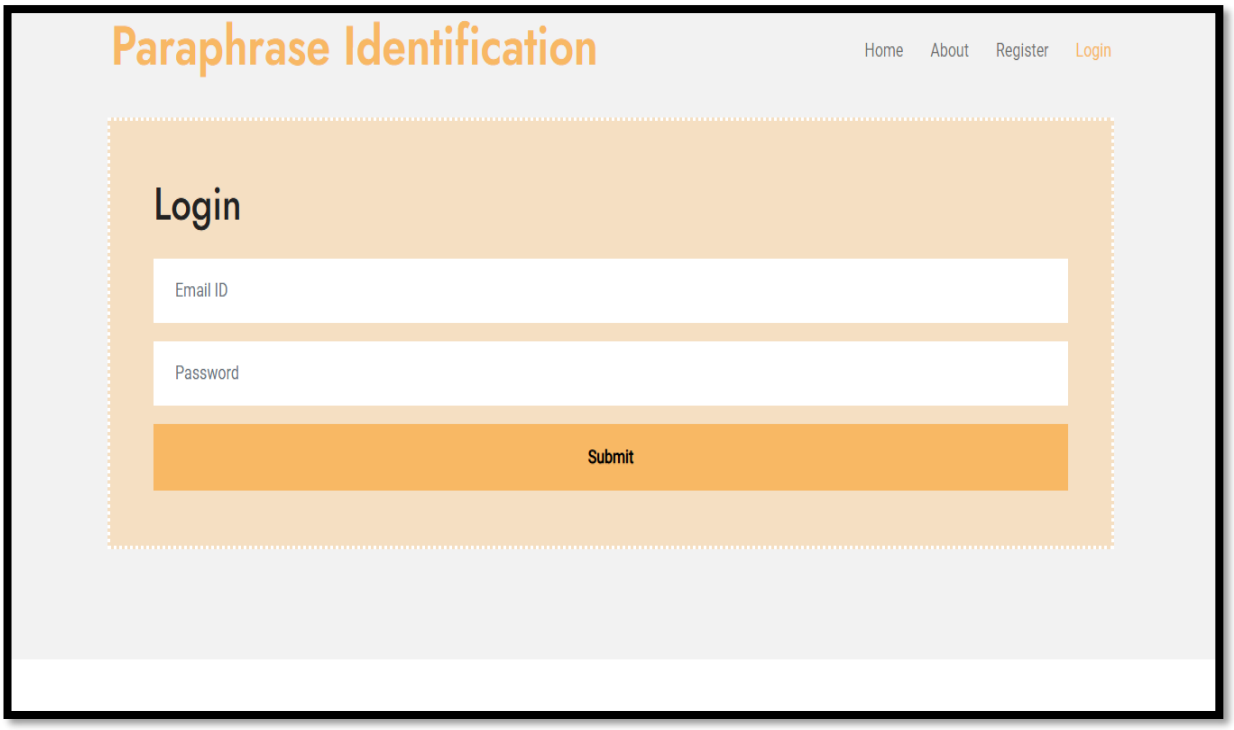


FIG 8.3: LOGIN PAGE

The registered users can access the system through the login page which is secure. Users must use valid usernames and passwords. The system cross checks the details typed and allows access only when the credentials are valid. Users are shown error messages when they make invalid logins so that they can be guided accordingly. This will provide security of data and help eliminate unauthorized access. The interface has been made user-friendly and easy to log in. It will be a portal to the core features of the system. The authentication will enhance reliability and trust in the application

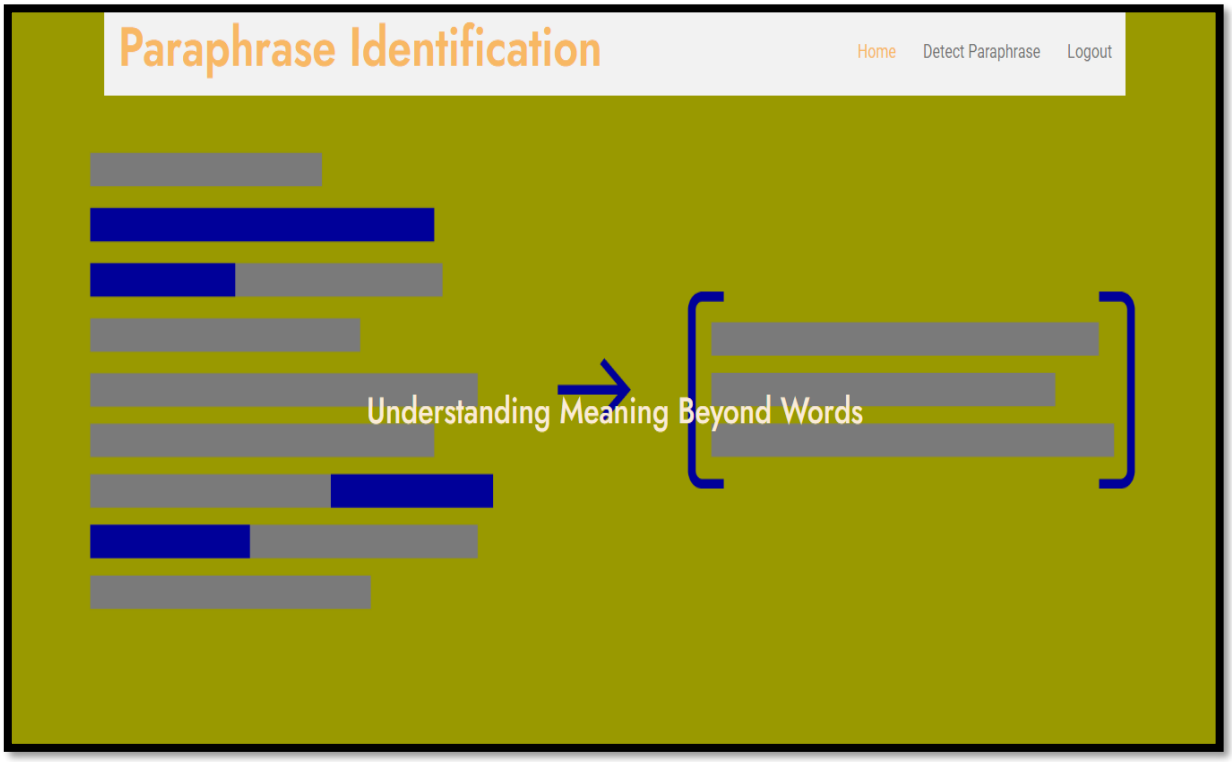


FIG 8.1 : USER HOME PAGE

On successful login, the user home page is shown and serves as the primary dashboard to the user. It offers navigation services to reach various system functions. At this page, users can move to the prediction module easily. The interface is made clean and easy to use. It also makes sure that the users learn fast on how to use the system. Critical moves and connections are well in sight. This page benefits the interaction and accessibility of the user. It can be used as a control panel to carry out system operations.



FIG 8.5 : PREDICTION PAGE

The functional part of the system is the prediction page. It gives the option of adding two sentences to be paraphrased. These inputs are executed by deep learning models which are trained in the system. The page has input fields that allow users to easily key in their sentences. After this is sent, the system compares the semantic similarity of the sentences. The page is made simple and efficient in order to have quick interaction. It is important in illustrating how the model works. It has a user-friendly interface that promotes easy addition and processing of data.

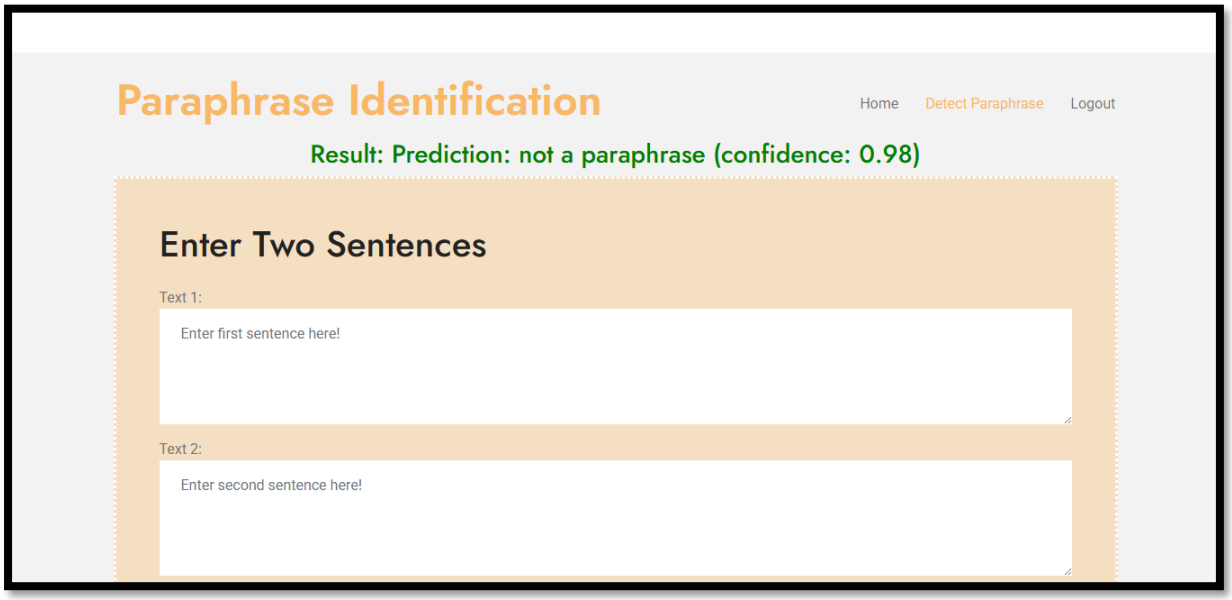


FIG 8.6 RESULT PAGE

The output of the paraphrase identification process is given in the result page. The system then classifies the input sentences as either paraphrase and non-paraphrase after processing the sentences. This outcome is presented in a clear way in order to make it easy to comprehend by the user. On this page, we can get instant feedback of the model prediction. It assists the users to confirm that the sentences are alike in meaning. The result is in an easy-to-read format. This improves user satisfaction and usability of the system. It closes the loop of the system.

CHAPTER 9

CONCLUSION

AND

FUTURE ENHANCEMENTS

9.CONCLUSION

This project on Paraphrase Identification successfully demonstrates the process of classifying whether two sentences convey the same meaning. By experimenting with various deep learning algorithms such as CNN, RNN, and BERT, the study establishes a solid baseline for sentence similarity detection. The exploration of advanced transformer-based models like RoBERTa further highlights the advantages of leveraging pre-trained language representations and contrastive learning techniques to capture semantic nuances more effectively. The comparative analysis of traditional and proposed models reveals that transformer architectures outperform classical neural networks in understanding complex sentence relationships. and RoBERTa, in particular, deliver superior results by learning rich contextual embeddings that improve classification accuracy. These findings confirm the growing importance of large-scale pre-training and fine-tuning in natural language tasks. Though this project is primarily educational and does not target real-time applications, it provides valuable insights into the design and implementation of state-of-the-art paraphrase detection systems. The methodology and results serve as a foundation for further research and development in semantic textual similarity, question answering, and related fields. Overall, the project strengthens understanding of semantic modeling techniques and their practical implications in natural language processing.

FUTURE ENHANCEMENT

The existing system of paraphrase identification shows the successful implementation of deep learning and transformer-based models to identify semantic similarity between sentence pairs. Nevertheless, the system may be also improved and expanded in several areas to make it more efficient, convenient and applicable in the real world.Among the major future improvements, one can distinguish incorporation of larger and more diverse data such as multilingual data. This would enhance the generalization of the model when using various language and fields to make the system more robust and wide-ranged. Also, more sophisticated transformer models like GPT or T5 can be also used to increase accuracy and attain more contextual insight.The other notable improvement is the introduction of real-time processing capabilities. As it is, the system will operate in a controlled setting, but can be scaled to support real-time user queries in a way that will be advantageous to implement the system in applications such as chatbots, virtual assistants, and search engines.The system may also be enhanced with the functionality of explainability, in which the model will give reasons or indicate significant words that had an effect on its prediction. This will make the users more transparent and trusting in the system as they can know how the system makes its decisions.

Moreover, it is possible to develop more interactive and responsive user interface to improve user experience. The system can be made easier to use and more interactive by adding features like sentence suggestions, auto-corruption and showing similarity scores. The other aspect that can be enhanced is the optimization of the model to create a more efficient and lower cost model. Such methods as model compression, pruning, and effective fine-tuning could be used to make the system quicker and fit the devices with low resources.Lastly, more NLP functions like text summarization, question answering and plagiarism detection can be added to the system, hence making it more operative and useful in practice. Altogether, these improvements can make a big contribution to the efficiency, scalability, and applicability of the system in reality.

1. Integrate more diverse datasets to improve the model's generalization across different languages and domains.
2. Explore transformer models with larger architectures like GPT or T5 for potentially better paraphrase detection accuracy.

3. Implement data augmentation techniques to increase the variety of training examples and reduce overfitting.
4. Develop an interactive user interface to allow easy input and real-time feedback on paraphrase detection.
5. Incorporate explainability methods to help understand why the model classifies sentence pairs as paraphrases or not.

REFERENCES

1. **Tianyu Gao, Xingcheng Yao & Danqi Chen (2021).** *: Simple Contrastive Learning of Sentence Embeddings.* arXiv. Introduces contrastive learning for high-quality sentence embeddings. [
2. **Yao Dou, Chao Jiang & Wei Xu (2022).** *Improving Large-scale Paraphrase Acquisition and Generation.* arXiv. Presents the MultiPIT Twitter corpus and SOTA benchmarks.
3. **Chao Zhou et al. (2022).** *Paraphrase Identification with Deep Learning: A Review of Datasets and Methods.* arXiv. Offers comprehensive survey and typology for paraphrase tasks.
4. **Rachna Jain et al. (2022).** *ParaCap: Paraphrase Detection Model Using Capsule Network. Multimedia Systems*. Proposes EPIC capsule architectures for paraphrase detection.
5. **D. Aravinda Reddy, M. Anand Kumar & K.P. Soman (2019).** *LSTM Based Paraphrase Identification Using Combined Word Embedding Features.* Springer. Applies LSTM with multilingual embeddings for Telugu paraphrase identification.
6. **Zhou & Bhat et al. (2020).** *A Deep Paraphrase Identification Model Interacting Semantics with Syntax. Complexity*. Integrates R-GCN with BERT-token to capture syntax and semantics.
7. **Xiaoyan Li et al. (2021).** *Short Text Paraphrase Identification Model Based on RDN-MESIM. Computational Intelligence and Neuroscience*. Uses deep semantic interaction on noisy short texts.
8. **Ko et al. (2020).** *Twice Fine-tuning Deep Neural Networks for Paraphrase Identification. Electronics Letters*. Demonstrates improved MRPC performance via successive BERT fine-tuning.
9. **A.B. Siddique, S. Oymak & V. Hristidis (2020).** *Unsupervised Paraphrasing via Deep Reinforcement Learning.* arXiv. Introduces PUP algorithm combining VAE and policy learning.
10. **He et al. (2019).** *LSTM Based Paraphrase Identification…* Soft Computing and Signal Processing. Uses Word2Vec, GloVe, fastText embeddings in LSTM for Telugu paraphrase tasks.

Pandu Raju

Nagaharshita-Saisireesha.docx

DSai Sireesha
AIDS
ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

Document Details

Submission ID
trn:oid:::1:3520918624

Submission Date
Mar 29, 2026, 10:53 PM GMT+5:30

Download Date
Mar 29, 2026, 10:56 PM GMT+5:30

File Name
Nagaharshita-Saisireesha.docx

File Size
1.7 MB

60 Pages

10,126 Words

59,411 Characters

30% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

171 Not Cited or Quoted 30%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

18% Internet sources
10% Publications
22% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Gmail harshu shetty <harshushetty1206@gmail.com>

IEEE ICSFT 2026 - ACCEPTANCE NOTIFICATION

Microsoft CMT <noreply@msr-cmt.org> Fri, Dec 12, 2025 at 11:39 AM
To: Naga Harshitha Gadamsetty <harshushetty1206@gmail.com>

Dear Naga Harshitha Gadamsetty

Paper ID / Submission ID : 651

Title : A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques

We are pleased to inform you that your paper has been accepted for the Oral Presentation as a full paper for the- “IEEE International Conference on Smart Futuristic Technology (ICSFT), will be held in Ghousia College of Engineering , Bengaluru, Karnataka, India.

All accepted and presented papers will be submitted to IEEE Xplore for the further publication and will be indexed by Ei Compendex and Scopus Indexing.

Complete the Registration Process (The last date of payment is 14 DEC 2025)

Registration Link for Payment :

For Indian Authors : https://rzp.io/rzp/RRkIlbco

For Foreign Authors :Given Soon

After the last date of registration IEEE PDF Express and E copyright information will be given for registered authors.

Note :

1. Any changes with the Author name, Affiliation and content of paper will not be allowed after acceptance. (if not added then can add and update in CMT)
2.This is Hybrid Conference, both online and physical presentation mode is available,

The reviews are below.

======= Review 1 =======

*** Relevance and timeliness: Rate the importance and timeliness of the topic addressed in the paper within its area of research.

Good (4)

*** Technical content and scientific rigour: Rate the technical content of the paper (e.g.: completeness of the

analysis or simulation study, thoroughness of the treatise, accuracy of the models, etc.), its soundness and scientific rigour.

Valid work but limited contribution. (4)

*** Novelty and originality: Rate the novelty and originality of the ideas or results presented in the

paper.

Some interesting ideas and results on a subject well investigated. (3)

*** Quality of presentation: Rate the paper organization, the clearness of text and figures, the completeness and accuracy of references.

Well written. (4)

*** Strong aspects: Comments to the author: what are the strong aspects of the paper?

In this paper, a new application is proposed. In addition, the paper discusses the different aspects of technology in the field of Programming testing to guarantee programming quality

*** Weak aspects: Comments to the author: what are the weak aspects of the paper?

*** Recommended changes: Please indicate any changes that should be made to the paper if accepted.

The paper should be more result oriented.

======= Review 2 =======

*** Relevance and timeliness: Rate the importance and timeliness of the topic addressed in the paper within its area of research.

Acceptable (3)

*** Technical content and scientific rigour: Rate the technical content of the paper (e.g.: completeness of the

Analysis or simulation study, thoroughness of the treatise, accuracy of the models, etc.), its soundness and scientific rigour.

Valid work but limited contribution. (3)

*** Novelty and originality: Rate the novelty and originality of the ideas or results presented in the paper.

Some interesting ideas and results on a subject well investigated. (3)

*** Quality of presentation: Rate the paper organization, the clearness of text and figures, the completeness and accuracy of references.

Readable, but revision is needed in some parts. (3)

*** Strong aspects: Comments to the author: what are the strong aspects of the paper

The paper reviews different aspects of the concern

*** Weak aspects: Comments to the author: what are the weak aspects of the paper?

The future contribution of the paper should be mentioned in the paper

The presentation quality of the paper has to be improved significantly. The details of the author's contribution are too little.

*** Recommended changes: Recommended changes. Please indicate any changes that should be made to the paper if accepted...

The presentation quality of the paper has to be improved significantly

The detail of simulation/Result is too little.

======= Review 3 =======

*** Relevance and timeliness: Rate the importance and timeliness of the topic addressed in the paper within its area of research.

Good (4)

*** Technical content and scientific rigour: Rate the technical content of the paper (e.g.: completeness of the

analysis or simulation study, thoroughness of the treatise, accuracy of the models, etc.), its soundness and scientific rigour.

Marginal work and simple contribution. Some flaws. (2)

*** Novelty and originality: Rate the novelty and originality of the ideas or results presented in the paper.

Minor variations on a well investigated subject. (2)

*** Quality of presentation: Rate the paper organization, the clearness of text and figures, the completeness and accuracy of references.

Okay. Well Written

*** Strong aspects: Comments to the author: what are the strong aspects of the paper

In this research, a new scenario of the technology has been introduced; research may be interesting for readers in this area.

*** Weak aspects: Comments to the author: what are the weak aspects of the paper?

1. The presentation should be substantially improved.

2. The presentation of the proposed algorithm is too conceptual, and details of how to operate the proposed algorithm in practice should be clearly elaborated.

3. All formulas must well written instead of copy paste of image.

Note: No major revisions in paper as per reviews and Similarity index is fine.

Thanks & Regards
TPC Chair
IEEE ICSFT 2026
Email Support : conficsft@gmail.com
Mob No : +91 8805435941
Website : https://icsft.in/

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

2026 International Conference on Smart Futuristic Technology (ICSFT)

2nd – 3rd January, 2026

Certificate

This is to certify that Dr./Prof./Mr./Ms. ***N Venkata Subbaiah*** *has presented paper entitled* ***A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques*** *in 2026 International Conference on Smart Futuristic Technology (ICSFT) during 02nd to 3rd January 2026.*

Dr. N S Kumar
General Chair

Dr. Dilshad Begum
Organizing Chair

2026 International Conference on Smart Futuristic Technology (ICSFT)
Bengalore, India. Jan 2-3, 2026

A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques

D Sai Shireesha
Assistant Professor
Department of AI&DS
Annamacharya University
Rajampet, India
desettyshireesha@gmail.com

Pandurangaraju B
Assistant Professor
Department of AI&DS
Annamacharya University
Rajampet, India
balaraju.pandu@gmail.com

G Naga Harshitha
Department of AI&DS
Annamacharya Institute of Technology and Sciences
Rajampet, India
harshushetty1206@gmail.com

G Vedavathi
Department of AI&DS
Annamacharya Institute of Technology and Sciences
Rajampet, India
guduruvedavathi@gmail.com

B Naga Tharun Kumar
Department of AI&DS
Annamacharya Institute of Technology and Sciences
Rajampet, India
tharunbanda1@gmail.com

N Venkata Subbaiah
Department of AI&DS
Annamacharya Institute of Technology and Sciences
Rajampet, India
subbunookala8688@gmail.com

***Abstract*— Among the tasks that are urgent in Natural Language Processing is the identification of paraphrase, which is a task that entails deciding whether two sentences have the same meaning despite the difference in wording. This project discusses the models of the deep learning in the classification of two sentences as either paraphrases or non-paraphrases. To determine the base performance, initially, the classic models, such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT will be used. Next to enhance the additional semantic insight and the precision, more sophisticated models employing the use of transformers including RoBERTa and contrastive learning are presented. The models are ready and tested on benchmark sets with an objective of sentence-pair classification. The strengths and weaknesses of each of the methods of resolvising the problem of syntax difference, ambiguity, and context-retainment are identified in this paper. These results can be used in the improvement of the following tasks: duplicate detection, question answering, and content summarization as they offer information on the successful paraphrase recognition in the context of the modern deep learning trends.**

Keywords—Paraphrase Identification, Natural Language Processing, Deep Learning, Sentence Similarity, Transformer Models, RoBERTa

I. INTRODUCTION

Detection of paraphrase is a key problem in Natural Language Processing (NLP) that seeks to determine whether two sentences are essentially conveying an equivalent meaning despite the fact that they may vary in the selection of words or the form of sentence. Of course, the same thought can be expressed differently using language and hence this is not an easy matter to learn semantic equivalence in machines. Given that traditional rule-based or lexical matching methods may not always be able to absorb the wider context and subtle nuances between sentences, it is especially true when sentences have complex structure or implication.This project will tackle the paraphrase identification problem as a supervised classification problem whereby the task will be to classify sentence pairs as paraphrases or nonparaphrases. It talks about different models of deep learning, including the traditional models of The convolutional neural networks (CNN) and reciprocal neural networks (RNN) where the former and the latter respectively assume local and sequential dependence to learn semantics, to transformer-based models like BERT, RoBERTa apply contextual embeddings to learn semantics better.

The CNNs can detect important phrases using word embedding convolution filters and RNNs can retain the memory of the previous words as it understands the sentence flow. However, this kind of model may be susceptible to long-range interactions or it may be very large and slow to train to generalize. BERT is a powerful model, since it used both-way transformers to extract the context in both directions, yet it involves a fine-tuning on each of the tasks. RoBERTa, also, takes a step further to assist in simplifying the training processes, as well as utilization of larger data sets; this results in better generalization.

These models are trained and tested using the benchmark datasets and taking into consideration such measures as accuracy, F1-score, and so on. It attempts to understand how different architectures can overcome such problems like ambiguous expressions, syntactic variability and semantic similarity. Through these comparisons, the project assists in the development of techniques of paraphrases that may be applied in systems in the case of accurate systems such as in question answering, detecting and developing chatbots and similar systems.

II. LITERATURE SURVEY

Identification of paraphrases has been the topic of intense interest in the last few years because of its uses in machine translation, question answering and in detecting plagiarism. The initial works have used syntactic similarity and statistical techniques, but recent developments in deep learning have significantly enhanced the performance.

Contrastive learning is one of the most powerful methods that promote sentence representation through the separation of similar and dissimilar pairs. Gao et al. [1] proposed a low cost contrastive learning network, which generates high quality sentence embeddings, and is much more competitive at that point than earlier supervised models with less annotation.

Dou et al. [2] were another major contributor, as they focused their work on large-scale paraphrase generation, and their contribution is that they created the MultiPIT dataset, based on Twitter, and set state-of-the-art (SOTA) baselines on pretrained transformers.

979-8-3503-5707-3/26/$31.00 ©2026 IEEE

A comprehensive overview of current paraphrase datasets and deep learning methods was provided by Zhou et al [3], who have presented a typology that categorises models in terms of architecture and purpose functionalities. This literature gives a basis on which the development of paraphrase detection systems can be analyzed.

New architectures such as Capsule Networks have been put to use as well. A model, ParaCap, emerged by Jain et al. [4], where the generalization of multiple benchmarks is improved through the use of improved capsule structures to maintain spatial hierarchy in feature learning.

Recurrent models, especially LSTMs, are not no longer relevant. Related LSTMs Reddy et al. [3] 5engaged the multilingual embeddings to detect Telugu paraphrase in LSTMs and demonstrated that the performance is better with language-specific embeddings. Likewise, He et al. [4] 10 also tested the various embedding methods (Word2Vec, GloVe, fastText) in LSTM networks to enhance semantic matching.

Zhou and Bhat [5] 6 investigated syntax-conscious methods, and they used Relational Graph Convolutional Networks (RGCN) with BERT to work together to capture syntax and semantics. Li et al. [6] 7suggested a powerful approach to noisy short text processing with deep semantic interaction networks.

That repetitive fine-tuning of BERT (twice fine-tuning) can dramatically increase accuracy on benchmark datasets such as MRPC was demonstrated by Ko et al. [8]. Finally, Siddique et al. [7] 9 proposed PUP, an unsupervised paraphrasing model, which applies Variational Autoencoders (VAE) and deep reinforcement learning to learn paraphrasing without labeled data.

All of these studies indicate a move towards deep, contextualised systems that use supervised and unsupervised learning.

III. PROPOSED SYSTEM

The deep learning models, including CNN, RNN, BERT, and RoBERTa, will help approach the problem of paraphrase identification systematically with this project methodology. The entire working process may be divided into three major steps; data collection, data preprocessing, model training and evaluation.

A. Data Collection

The initial step is to 1 collect a strong dataset that can be used to identify paraphrases. Commonly used benchmark datasets, such as the Paraphrase Adversaries from Word Scrambling (PAWS) datasets, are associated with a high number of annotated pairs of sentences. There are sentence pairs in every dataset that are labeled as paraphrases (label = 1) and nonparaphrases (label = 0). The data may be formally expressed as:

$$D = \{(S_i^1, S_i^2, y_i) \mid i = 1,2,\ldots,N\}$$

where S_i^1 and S_i^2 denote the two sentences in the ith pair, and $yi \in \{0,1\}$ indicates whether the sentences are paraphrases.

B. Data Preprocessing

The important step is the preprocessing to make sure that the input text is clean and formatted to the models. The preprocessing pipeline consists of these sub-steps:

- **Cleaning:** It is also a step that includes turning all the characters into lower case so as to minimize the vocabulary.
- **Tokenization:** In the case of traditional models such as CNN and RNN, words are tokenized at the word level, whereas transformer models are based on subword tokenization with WordPiece or Byte Pair Encoding (BPE).
- **Stopword Removal and Lemmatization (optional):** Stopword Removal and Lemmatization (option): Although certain models can be helped by stopping words, word reduction to base form, transformer models do not usually need this feature since they have contextual embeddings.
- **Padding and Truncation:** Neural networks take inputs of constant length and thus shorter sentences are padded with zeros and longer sentences are truncated to a maximum length L. Formally, every tokenized sentence S is converted to a fixed-length sequence:

$$S = (w_1, w_2, \ldots, w_L)$$

where w_i represents a token or padding token if the sentence length is less than L.

- **Embedding Initialization:** For CNN and RNN, pretrained word embeddings like GloVe or Word2Vec are used to convert tokens into dense vectors $\mathbf{e} \in \mathbb{R}^d$, where d is the embedding dimension. Transformers inherently generate contextual embeddings during training.

C. Model Training

The primary concept of the methodology consists in training deep learning models which can distinguish between sentence pairs, termed as paraphrases and those termed otherwise. The models that have been trained and compared include:

1) Convolutional Neural Network (CNN).

CNNs derive local n-grams of word embeddings by convolutional filters. Combining a sentence embedding matrix, a convolution operation with a filter of some window size produces a feature map.

2) Recurrent Neural Network (RNN)

RNNs go through sequences sequentially, and maintain a hidden state that recalls the past. An input is embedded at every time step and an activation function performed. Variations that address vanishing gradient issues are also gating mechanisms like LSTM or GRU.

3) BERT and RoBERTa

Transformer-based models, like BERT and RoBERTa, have the multi-head self-attention characteristic to learn the global dependencies. Supposing that input tokens are included, the attention system computes weighted sums of values.

The model is applied to the two sentences in the scenario of sentence-pair classification, the outcome of the special [CLS] token being used as the aggregate representation.

4) Classification Layer

A fully connected layer that uses softmax operation classifies the sentence pair:

D. *Loss Function and Optimization*

The models are trained to minimize the cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

where yi is the true label and $\hat{y}i$ is the predicted probability of paraphrase.

Optimization is performed using Adam or its variants with learning rate scheduling and early stopping based on validation performance.

IV. RESULT AND DISCUSSION

The accuracy and recall of different paraphrase identification models based on deep learning were measured by such common classification measures as precision, recall, F1score, and accuracy. CNN, RNN, BERT, and RoBERTa were evaluated and trained on benchmark datasets.

The CNN model had a mean accuracy of 51, precision and recall were equal around 0.51 for the paraphrase and nonparaphrase classes. Although the CNN was good at capturing local syntactic properties, it could not deal with long-range dependencies and deeper semantic complexities and achieved moderate results (F1-score of about 0.50).

In the same way, the RNN model performed slightly better with an accuracy of 51 percent also, albeit with a little higher recall (0.53 non-paraphrases and 0.49 paraphrases). Although RNNs can work with sequential data, vanishing gradients and small context windows can have constrained their performance on this task.

Transformer based models, conversely, became much better. BERT and its contextual bidirectional embeddings were far much better at 95 percent accuracy than CNN and RNN. The two classes were always precisioned and recalled (~0.93 to 0.95) and therefore, the F1-score was approximately 0.94 to 0.95. This confirms the ability of BERT to effectively achieve semantic similarities and slight sentence variations.

These findings were further enhanced by RoBERTa that did 96 percent accuracy. It had the highest precision and recall rate on paraphrase and non-paraphrase classes and F1-scores of about 0.96. The generalization and more comprehensive semantic interpretation was achieved due to the powerful pretraining method and dynamic masking used by RoBERTa.

All in all, the results demonstrate that transformer based models particularly RoBERTa are far superior in the task of paraphrase identification in comparison to the standard CNN and RNN models. This demonstrates the importance of contextualized embeddings and more advanced pretrainers to recognize paraphrased text properly.

TABLE I. ROBERTA CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
Not Paraphrase	0.97	0.96	0.96	2758
Paraphrase	0.95	0.97	0.96	2183
Accuracy			**0.96**	4941
Macro Avg	0.96	0.96	0.96	4941
Weighted Avg	0.96	0.96	0.96	4941

TABLE II. BERT CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
Not Paraphrase	0.96	0.94	0.95	2758
Paraphrase	0.93	0.95	0.94	2183
Accuracy			**0.95**	4941
Macro Avg	0.94	0.95	0.95	4941
Weighted Avg	0.95	0.95	0.95	4941

TABLE III. RNN CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
Not Paraphrase (0)	0.52	0.53	0.52	4402
Paraphrase (1)	0.51	0.49	0.50	4330
Accuracy			**0.51**	8732
Macro Avg	0.51	0.51	0.51	8732
Weighted Avg	0.51	0.51	0.51	8732

TABLE IV. CNN CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
Not Paraphrase (0)	0.51	0.63	0.56	4402
Paraphrase (1)	0.51	0.39	0.44	4330
Accuracy			**0.51**	8732
Macro Avg	0.51	0.51	0.50	8732
Weighted Avg	.51	0.51	.50	8732

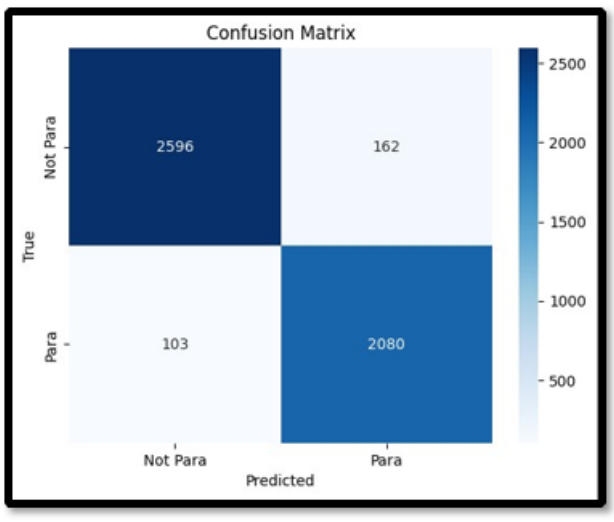


Fig. 1. Confusion matrix for BERT

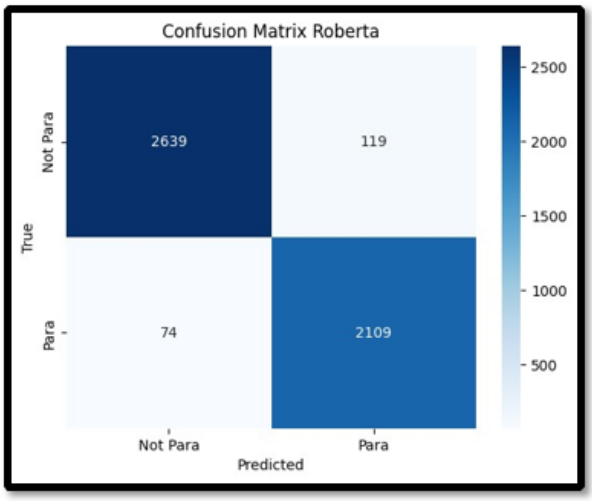


Fig. 2. Confusion matrix for RoBerta

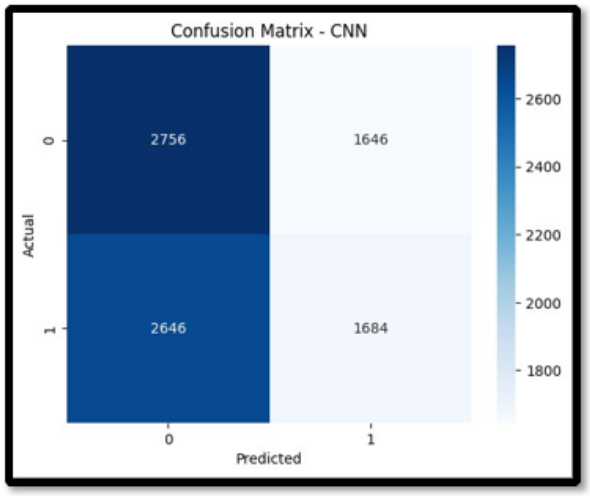


Fig. 3. Confusion Matrix CNN

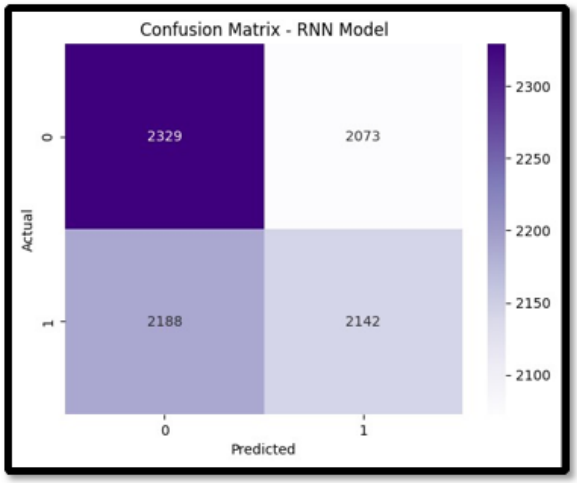


Fig. 4. Confusion Matrix RNN

V. CONCLUSION

It involved a comprehensive analysis of paraphrase recognition under various deep learning lines like the traditional models like CNN and RNN models to the new models that include the application of transformers like BERT and RoBERTa. The fundamental aim was to explore and compare the degree to which these models were in a position to differentiate between a couple of sentences as either paraphrases or non-paraphrases by comprehending semantic similarity despite the variations in expressions and make-up.

The experimental results indicate a clear suggestion that the classical models including CNN and RNN are unable to grasp the subtle meaning of sentence pairs as it is perceived by the relatively low accuracy and F1-scores of approximately 50%. These models are constrained in not being able to control effectively the long range dependencies and higher contextual information which is significant in the interpretation of paraphrasing in natural language.

Nonetheless, transformer models are superior to other models such as BERT and RoBERTa models, which increased their accuracy to over 95%. Their bilateral encoding of context, combined with their pretraining on large scale data corpora, allow such models to encode the fine details of the syntactic and semantic relations in a more robust way. RoBERTa of course with its optimized training techniques performed a bit better than BERT and this means that fine-tuning of the paraphrase-detecting performance can be enhanced by more sophisticated pretraining.

The current paper justifies the application of contextualized embeddings and transformers in NLP applications that require a high level of semantic interpretation. It also highlights potential merits of contrastive learning and transfer learning in improving generalization of the model in sentence-pair classification.

Overall, it is possible to speak about the findings of this work as informative regarding the capacities and weaknesses of the different deep learning models to identify paraphrases. Future research on enhancing accuracy and robustness could be achieved through future studies on hybrid methods, domain specific fine-tuning and on larger data sets.

REFERENCES

[1] T. Gao, X. Yao, and D. Chen, “SimCSE: Simple Contrastive Learning of Sentence Embeddings,” EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings, pp. 6894–6910, Apr. 2021, doi: 10.18653/v1/2021.emnlp-main.552.

[2] Y. Dou, C. Jiang, and W. Xu, “Improving Large-scale Paraphrase Acquisition and Generation,” Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, pp. 9301–9323, Oct.2022, doi: 10.18653/v1/2022.emnlp-main.631.

[3] D. Aravinda Reddy, M. Anand Kumar, and K. P. Soman, “LSTM based paraphrase identification using combined word embedding features,” Advances in Intelligent Systems and Computing, vol. 898, pp. 385–394, 2019, doi: 10.1007/978-981-13-33934_40.

[4] D. Aravinda Reddy, M. Anand Kumar, and K. P. Soman, “LSTM based paraphrase identification using combined word embedding features,” Advances in Intelligent Systems and Computing, vol. 898, pp. 385–394, 2019, doi: 10.1007/978-981-13-33934_40.

[5] L. Kong, Z. Han, Y. Han, and H. Qi, “A Deep Paraphrase Identification Model Interacting Semantics with Syntax,” Complexity, vol. 2020, 2020, doi: 10.1155/2020/9757032.

[6] J. Li, D. Zhang, and A. Wulamu, “Short Text Paraphrase Identification Model Based on RDN-MESIM,” Comput Intell Neurosci, vol. 2021, no. 1, p. 6865287, Jan. 2021, doi: 10.1155/2021/6865287.

[7] A. B. Siddique, S. Oymak, and V. Hristidis, “Unsupervised Paraphrasing via Deep Reinforcement Learning,” Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1800–1809, Jul. 2020, doi: 10.1145/3394486.3403231.

[8] Paraphrase Identification with Deep Learning: A Review of Datasets and Methods. arXiv. Offers comprehensive survey and typology for paraphrase tasks.

[9] ParaCap: Paraphrase Detection Model Using Capsule Network. Multimedia Systems. Proposes EPIC capsule architectures for paraphrase detection.

[10] Twice Fine tuning Deep Neural Networks for Paraphrase Identification. Electronics Letters. Demonstrates improved MRPC performance via successive BERT fine-tuning

ADDRESSING OF SUSTAINABLE DEVELOPMENT GOALS

TITLE OF THE PROJECT: A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques

S.No	SUSTAINABLE DEVELOPMENT GOALS	MAPPING	JUSTIFICATION
1.	No poverty	NA	
2.	Zero Hunger	NA	
3.	Good Health and Well-being	✓	Reduces workload and mental effort by automating text similarity checking.
4.	Quality Education	✓	Improves learning through better plagiarism detection systems. Ensures fair and accurate evaluation of academic content.
5.	Gender Equality	✓	Provides unbiased results using AI-based models. Eliminates human discrimination in evaluation.
6.	Clean water and Sanitation	NA	
7.	Affordable and Clean Energy	NA	
8.	Decent Work and Economic Growth	✓	Increases productivity by reducing manual text analysis work. Helps organizations save time and resources.
9.	Industry, Innovation and Infrastructure	✓	Uses advanced technologies like NLP and deep learning models. Supports innovation in AI-based text processing systems.
10.	Reduced Inequality	✓	Ensures equal and consistent evaluation for all users. Removes bias in content analysis.
11.	Sustainable Cities and Communities	✓	Supports smart applications like chatbots and search engine and improves access to accurate and relevant information.
12.	Responsible Consumption and Production	✓	Reduces duplication of content by detecting paraphrases and promotes efficient use of digital resources.
13.	Climate Action	NA	
14.	Life Below Water	NA	
15.	Life on Land	NA	
16.	Peace Justice and Strong Institutions	✓	Helps in plagiarism detection and content verification and Ensures transparency and integrity in systems.
17.	Partnerships for the Goals	NA	

SUPERVISOR **HEAD OF THEDEPARTMENT**

ANNAMCHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES: RAJAMPET

Department of Artificial Intelligence & Data Science

ASSESSMENT

Project Title: A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques

ABSTRACT

Paraphrase identification is a fundamental task in Natural Language Processing (NLP) that aims to determine whether two sentences express the same meaning using different wording. This classification-based project explores various deep learning architectures to classify sentence pairs into two categories: paraphrase or non-paraphrase. Initially, existing models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT are employed to benchmark performance. Subsequently, more advanced transformer-based models like and RoBERTa are proposed to enhance accuracy and semantic understanding. These models leverage pre-trained contextual embeddings and contrastive learning for improved textual similarity detection. The project uses benchmark datasets suitable for sentence-pair classification to train and evaluate the models. While this work is academic in nature, it simulates accurate challenges such as ambiguity, syntax variation, and context preservation. The goal is to understand the strengths and limitations of various models in paraphrase detection and to compare their performance comprehensively. This exploration can contribute to applications such as question-answer matching, duplicate detection, and content summarization systems.

Assessment (With Pos & PSOs)

Project Title	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
A Comprehensive Survey on Paraphrase Identification Using Deep Learning Techniques	3	3	3	3	3	1	1	3	3	2	2	3	3	3	2

Justification:

- **Time-Consuming Manual Evaluation**

Manual comparison of sentence similarity is time-consuming and inefficient. The proposed system automates paraphrase detection using deep learning models.

- **Reduction of Human Bias**

Manual evaluation of text similarity can vary between individuals. The system ensures consistent and unbiased results using AI models.

- **Handling Handwritten Variability**

Different sentence structures can express the same meaning. The system effectively captures semantic similarity using models like BERT and RoBERTa.

- **Semantic-Based Evaluation**

Unlike keyword matching, the system understands contextual meaning. It accurately detects paraphrases even with different wording..

- **Scalability for Large-Scale Exams**

The system can process large datasets efficiently. It is suitable for applications like plagiarism detection and search systems.

Signature of Coordinator **Head of the Department**